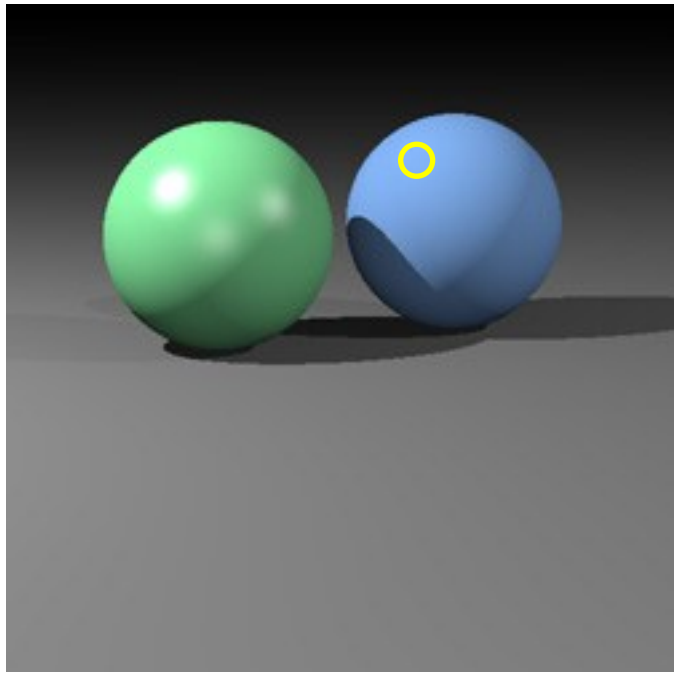
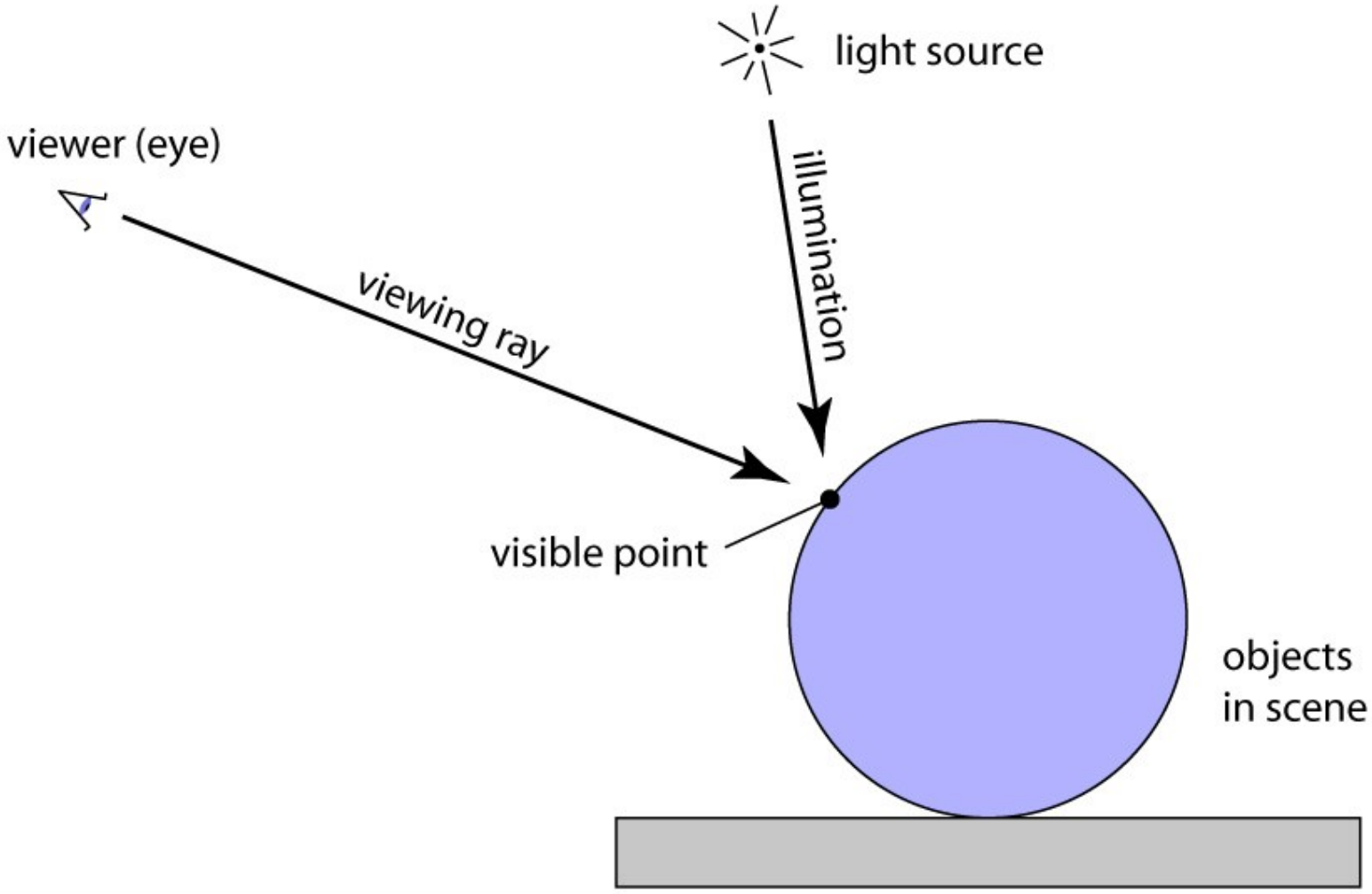
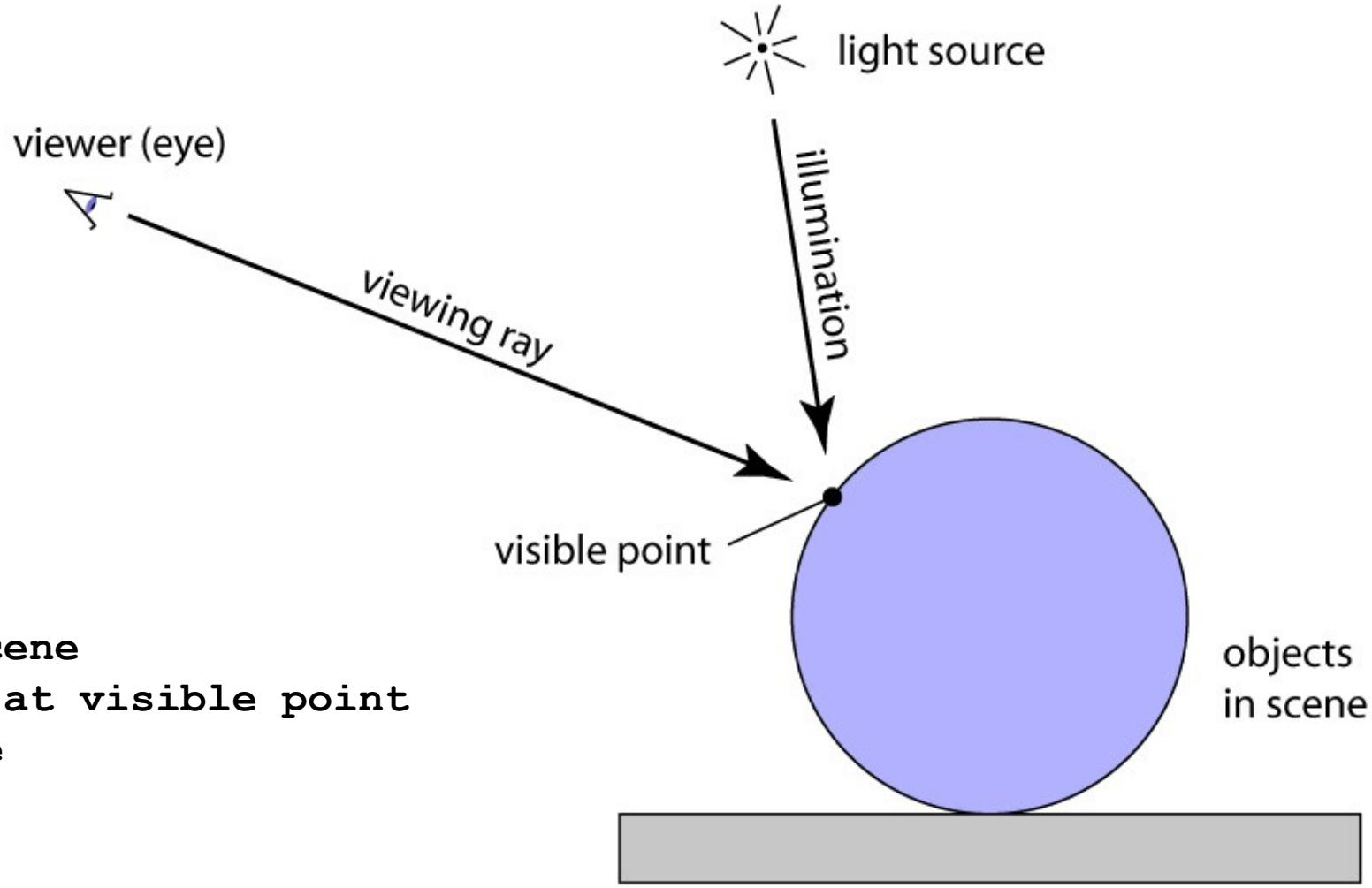
Ray Tracing

Ray tracing idea





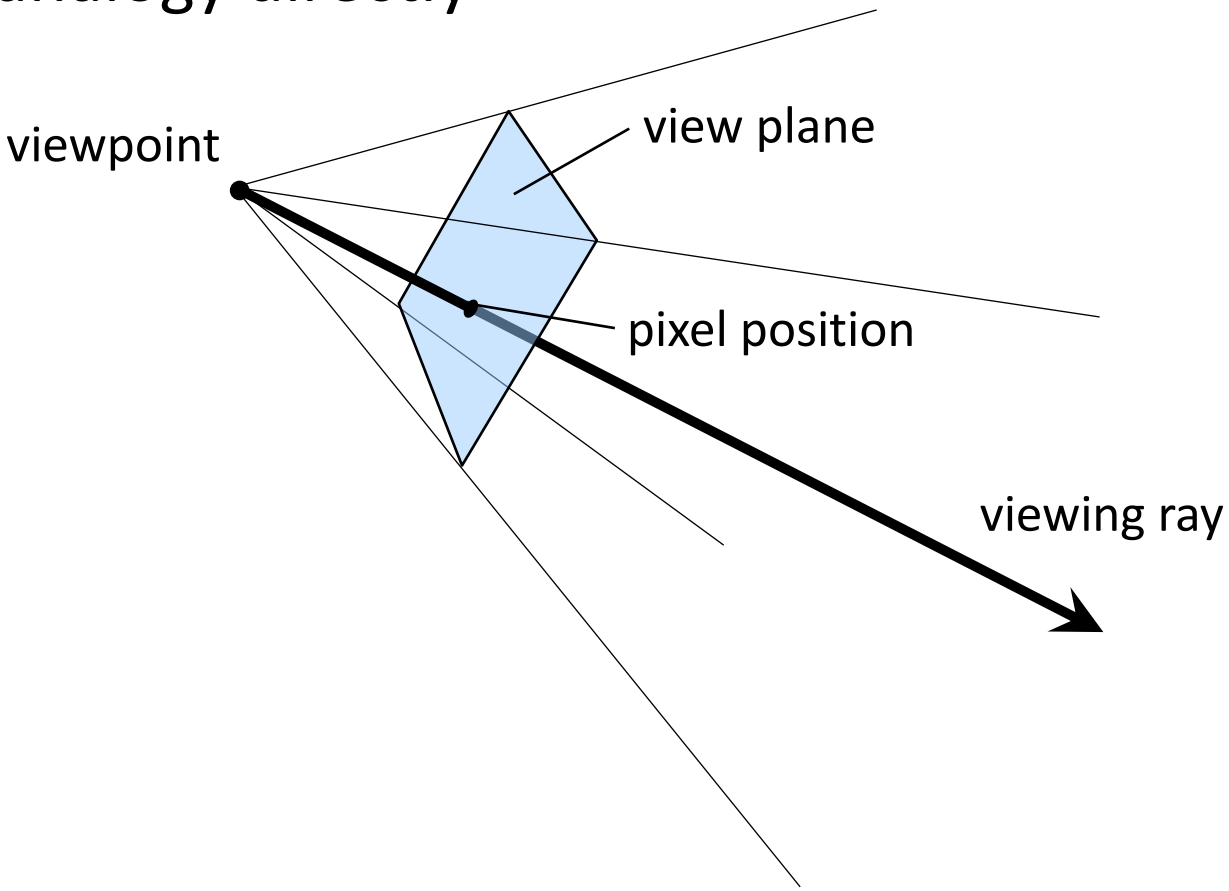
Ray tracing algorithm




```
for each pixel {
    compute viewing ray
    intersect ray with scene
    compute illumination at visible point
    put result into image
}
```

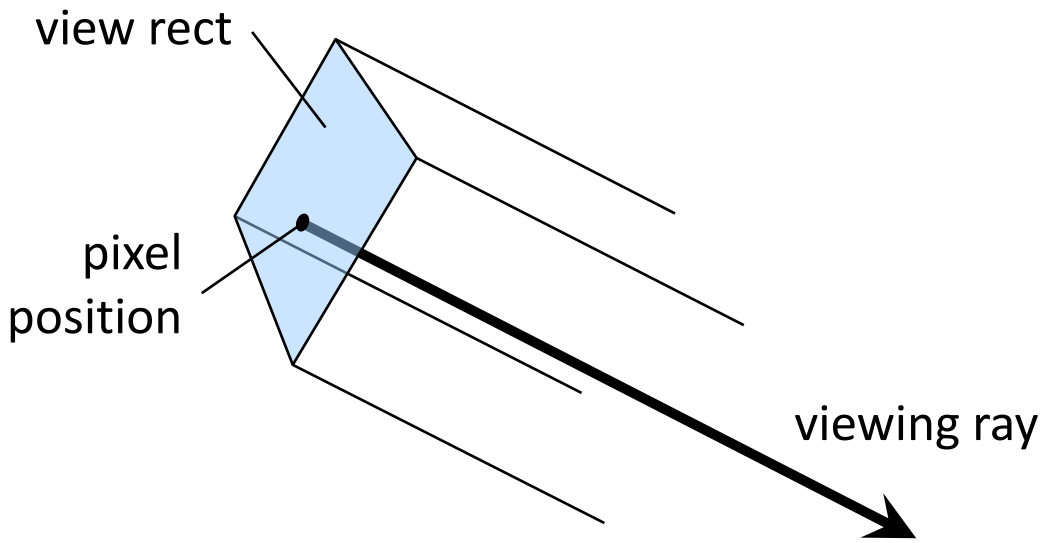
Generating eye rays

- Use window analogy directly

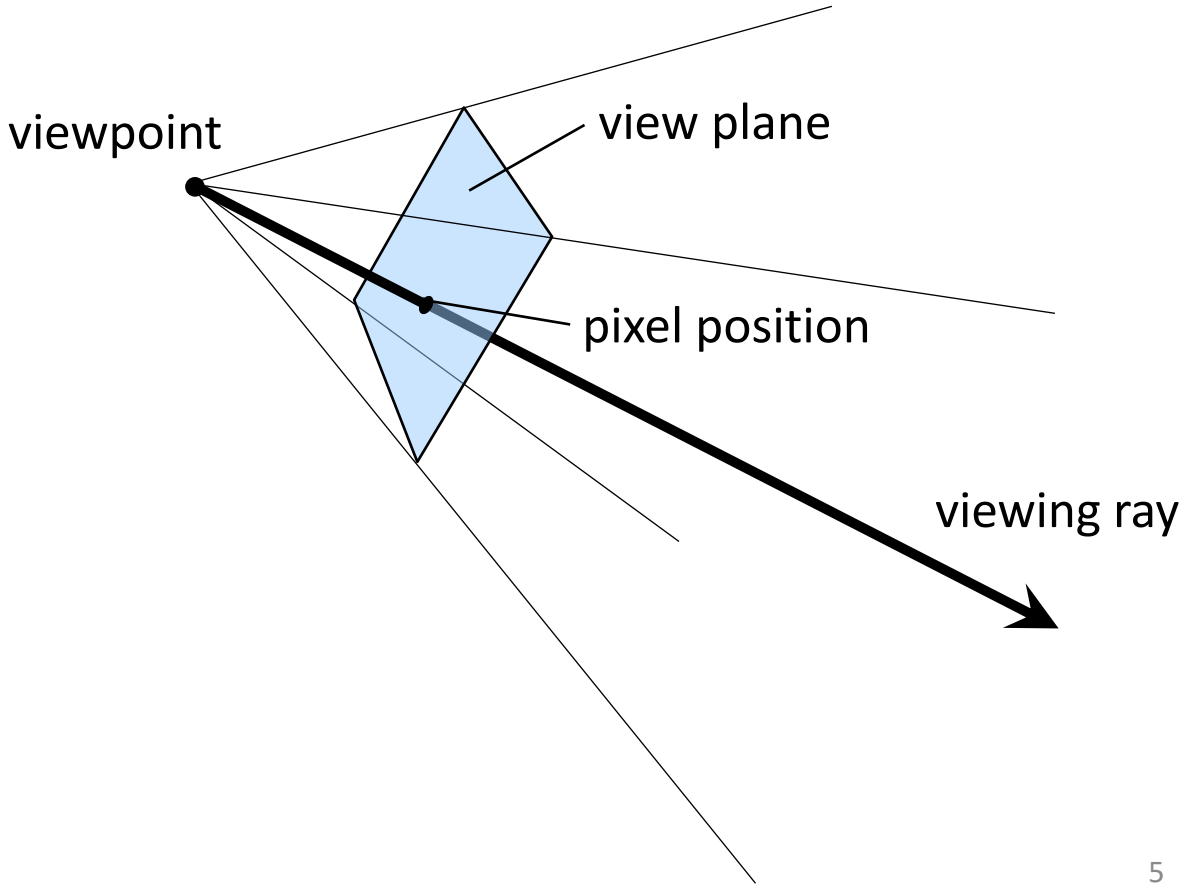


Generating eye rays

ORTHOGRAPHIC

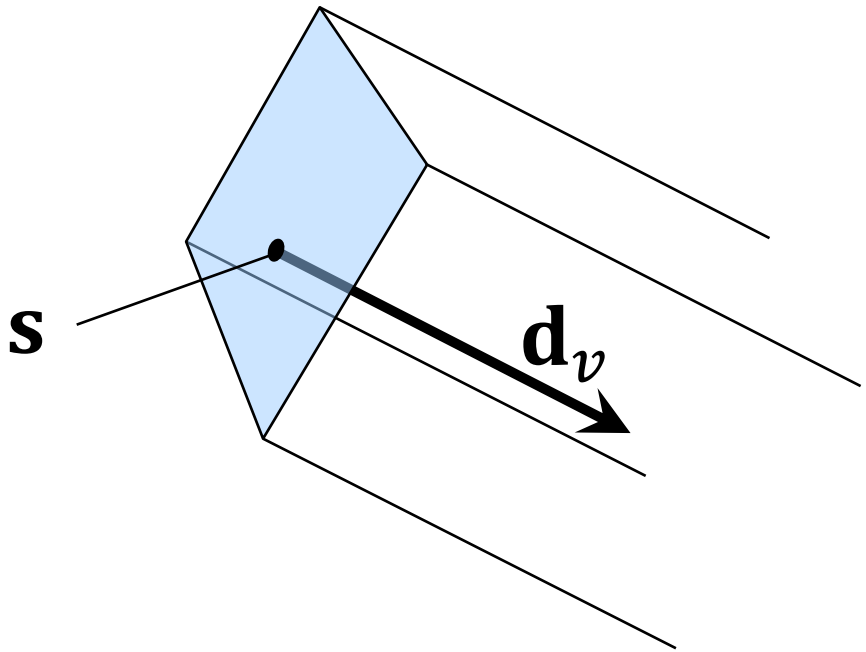


PERSPECTIVE



Generating eye rays—orthographic

- Just need to compute the view plane point $\mathbf{s}$



$$\mathbf{p} = \mathbf{s}$$
$$\mathbf{d} = \mathbf{d}_v$$
$$\mathbf{r}(t) = \mathbf{p} + t\mathbf{d}$$

But where exactly is the view rectangle?

Generating eye rays—orthographic

3D point (bold non-italic)

3D vector (bold non-italic)

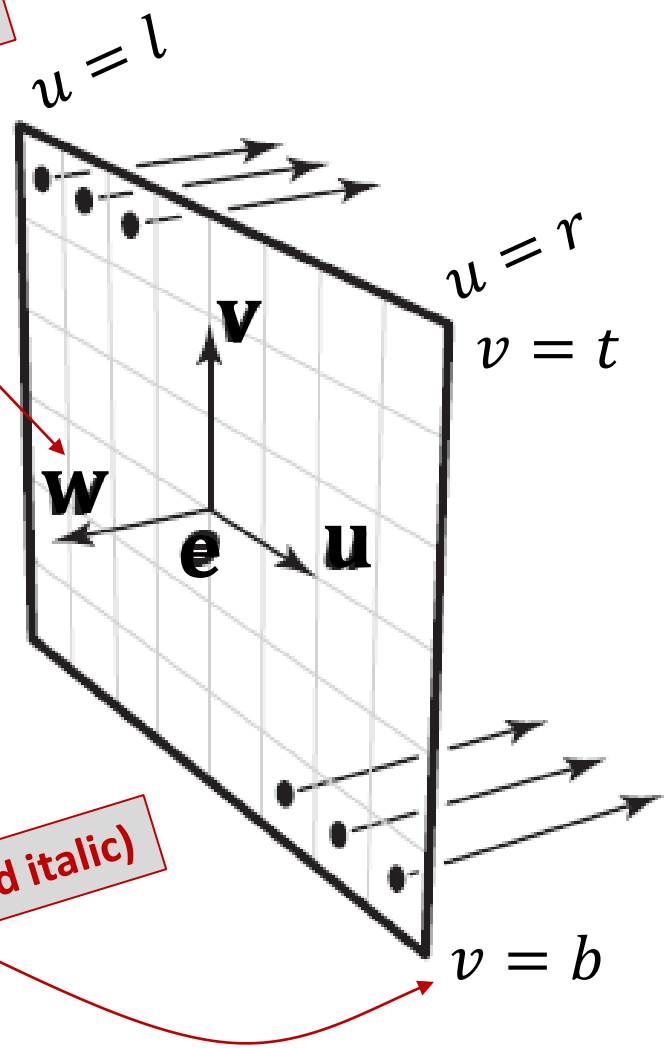
$$\mathbf{s} = \mathbf{e} + u\mathbf{u} + v\mathbf{v}$$

$$\mathbf{p} = \mathbf{s}$$

$$\mathbf{d} = -\mathbf{w}$$

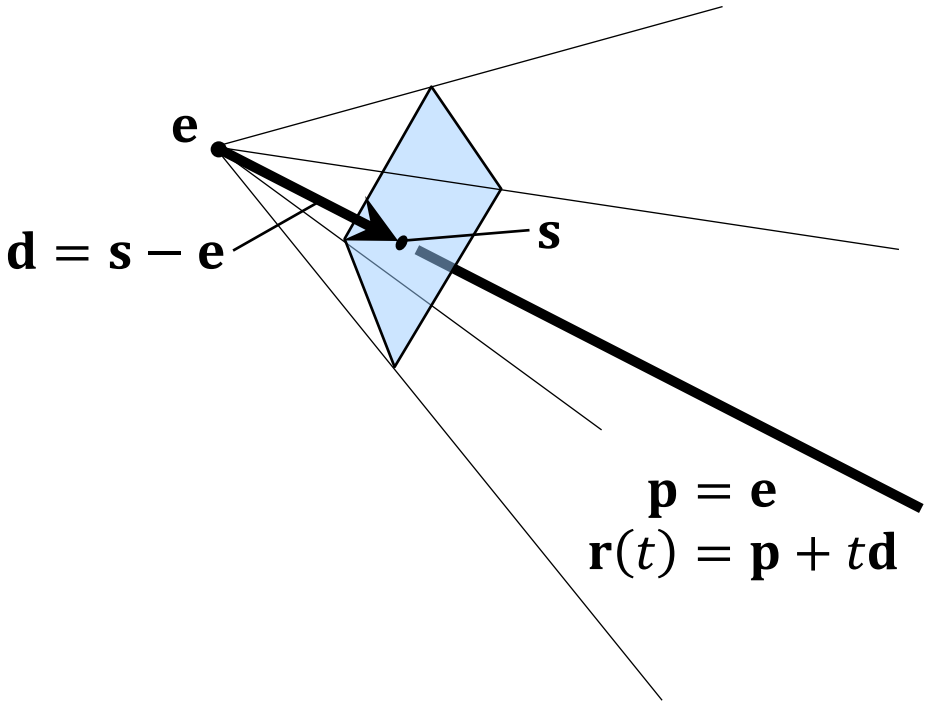
$$\mathbf{r}(t) = \mathbf{p} + t\mathbf{d}$$

Scalar (non-bold italic)



Generating eye rays—perspective

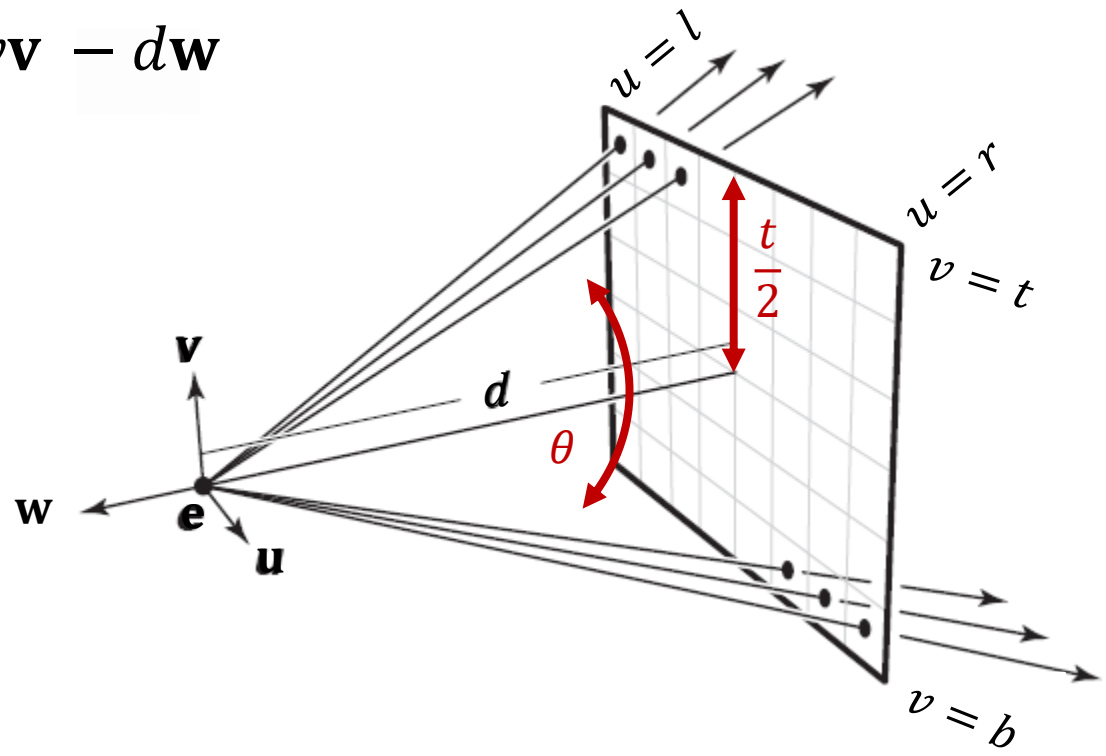
- View rectangle needs to be away from viewpoint
- Distance is important: “focal length” of camera
  - still use camera frame but position view rectangle away from viewpoint
  - ray origin always $\mathbf{e}$
  - ray direction now controlled by $\mathbf{s}$



Generating eye rays—perspective

- Compute $\mathbf{s}$ in the same way; just subtract $d\boldsymbol{w}$
  - coordinates of $\mathbf{s}$ are $(u, v, -d)$

$$\mathbf{s} = \mathbf{e} + u\mathbf{u} + v\mathbf{v} - d\mathbf{w}$$
$$\mathbf{p} = \mathbf{e}$$
$$\mathbf{d} = \mathbf{s} - \mathbf{e}$$
$$\mathbf{r}(t) = \mathbf{p} + t\mathbf{d}$$

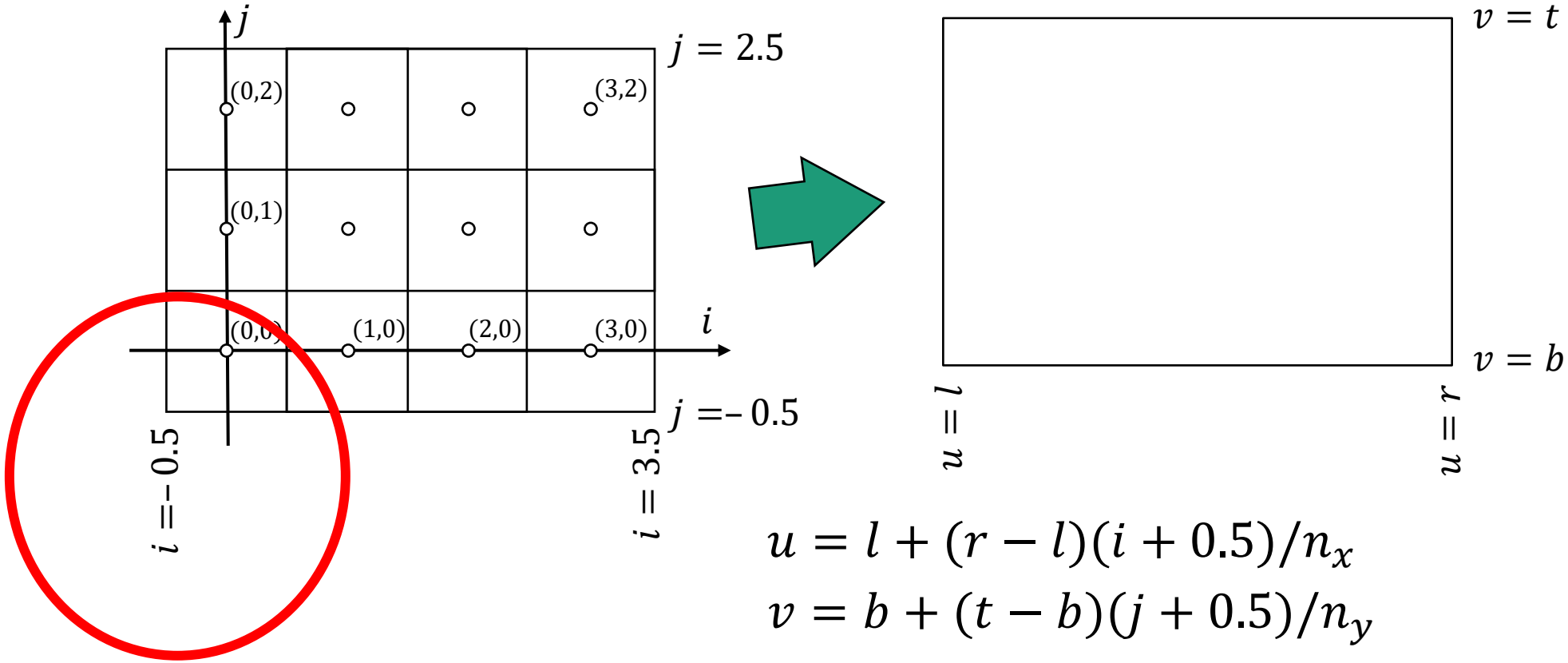


$$t = d \tan\left(\frac{\theta}{2}\right)$$

where θ is the field of view in the vertical direction

Pixel-to-image mapping

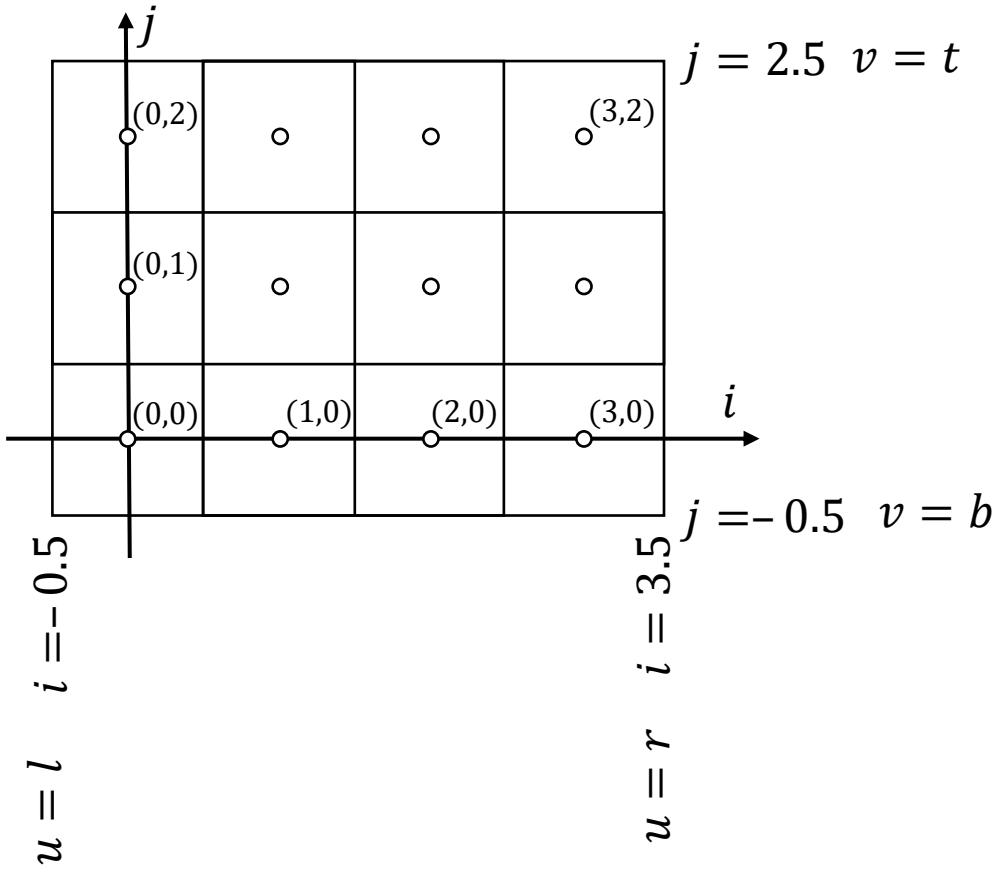
- One last detail: (u, v) coordinates of a pixel



$$u = l + (r - l)(i + 0.5)/n_x$$
$$v = b + (t - b)(j + 0.5)/n_y$$

Pixel-to-image mapping

- One last detail: (u, v) coordinates of a pixel

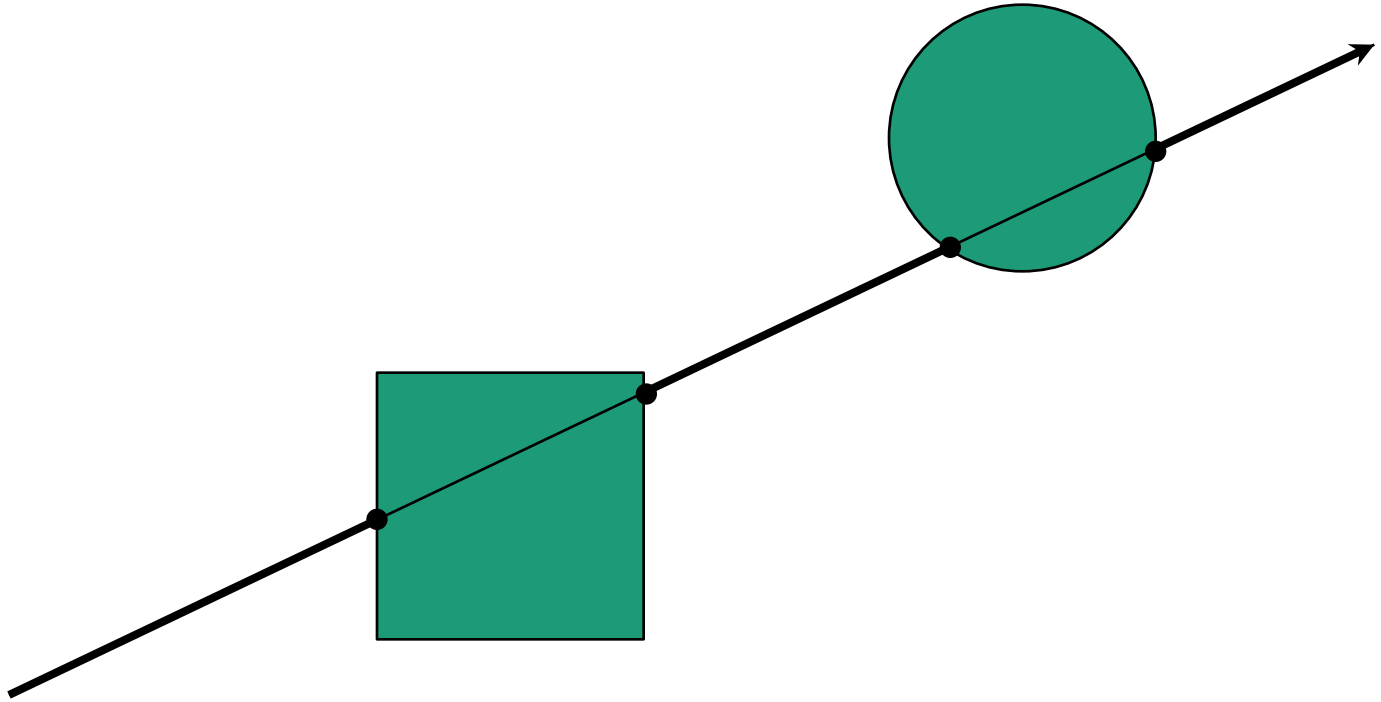


$$u = l + (r - l)(i + 0.5)/n_x$$
$$v = b + (t - b)(j + 0.5)/n_y$$

Generating eye rays – camera specification

- Given
  - Field of view in y direction,
  - eye point, look at, up direction
  - Image resolution $n_x \times n_y$
  - How do you generate eye rays?
  - Do you need to compute a viewing transformation matrix?
  - Do you need to invert a matrix?
  - What is the aspect ratio of the image?
  - What is the left right top and bottom?
  - If the focal length is unspecified, what should d be?

Ray intersection

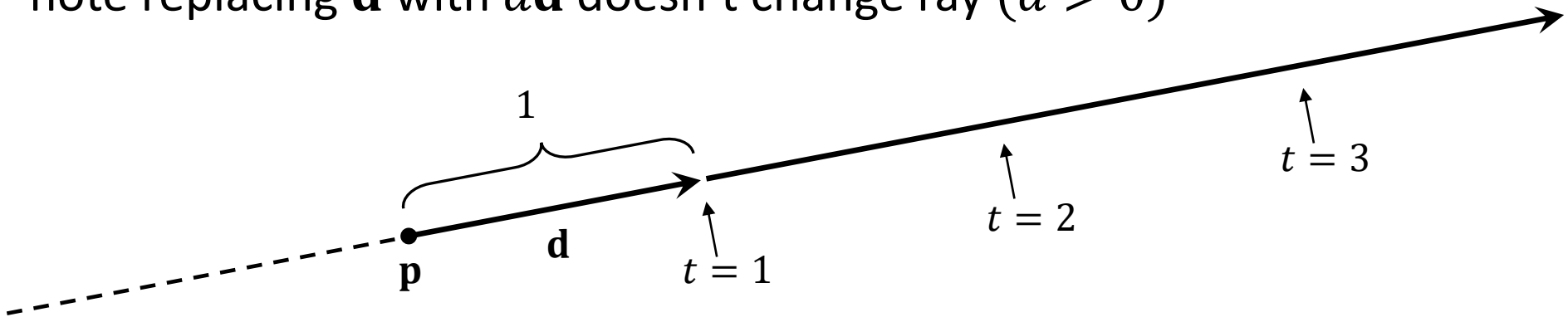


Ray: a half line

- Standard representation: point $\mathbf{p}$ and direction $\mathbf{d}$

$$\mathbf{r}(t) = \mathbf{p} + t\mathbf{d}$$

  - this is a parametric equation for the line
  - lets us directly generate the points on the line
  - if we restrict to $t > 0$ then we have a ray
  - note replacing $\mathbf{d}$ with $a\mathbf{d}$ doesn't change ray $(a > 0)$



Ray-sphere intersection: algebraic

- Condition 1: point is on ray

$$\mathbf{r}(t) = \mathbf{p} + t\mathbf{d}$$

- Condition 2: point is on sphere
  - assume unit sphere; see Shirley or notes for general

$$\|\mathbf{x}\| = 1 \;\Leftrightarrow\; \|\mathbf{x}\|^2 = 1$$
$$f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{x} - 1 = 0$$

- Substitute:

$$(\mathbf{p} + t\mathbf{d}) \cdot (\mathbf{p} + t\mathbf{d}) - 1 = 0$$

  - this is a quadratic equation in t

$at^2 + bt + c = 0$
where
$a = \mathbf{d} \cdot \mathbf{d}$
$b = 2\,\mathbf{p} \cdot \mathbf{d}$
$c = \mathbf{p} \cdot \mathbf{p} - 1$

Ray-sphere intersection: algebraic

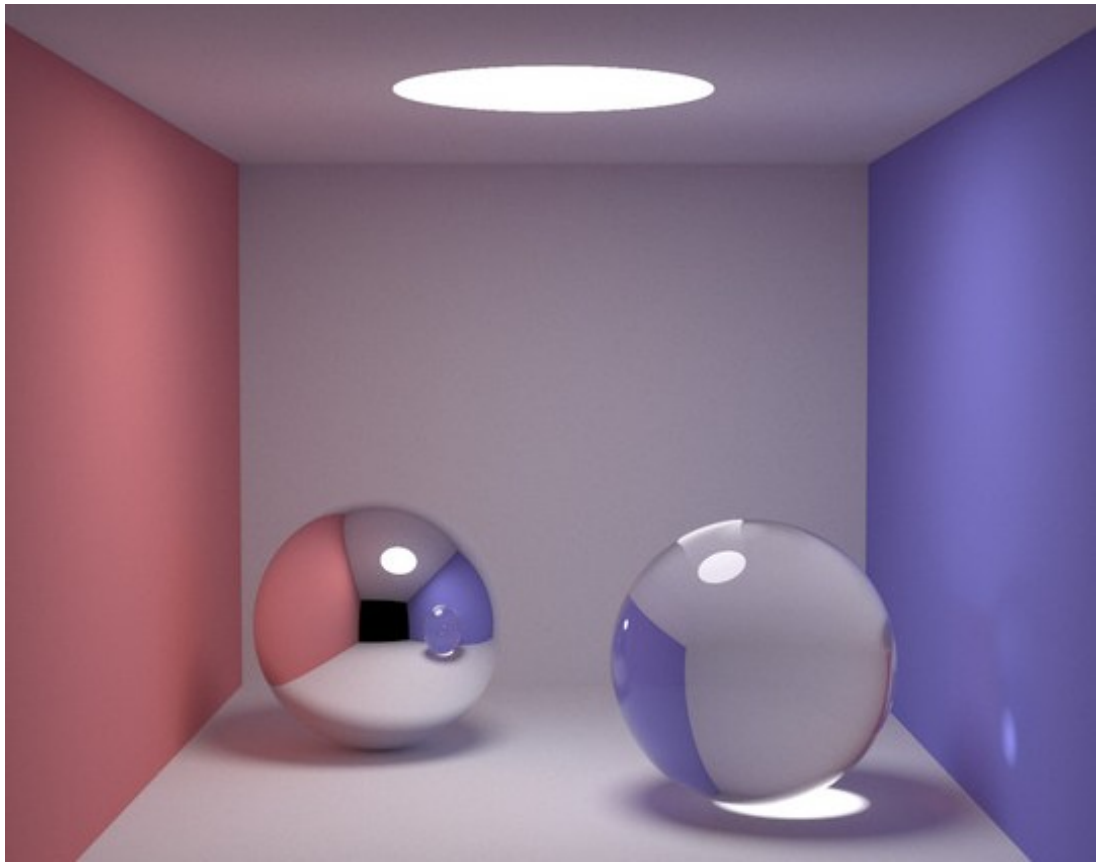
- Solution for t by quadratic formula

$$t = \frac{-\mathbf{d} \cdot \mathbf{p} \pm \sqrt{(\mathbf{d} \cdot \mathbf{p})^2 - (\mathbf{d} \cdot \mathbf{d})(\mathbf{p} \cdot \mathbf{p} - 1)}}{\mathbf{d} \cdot \mathbf{d}}$$

- Simpler form holds when $\mathbf{d}$ is a unit vector
  - We won't assume this in practice (because of ray transformations)

What can you do with Spheres? (ASIDE)

Physically based rendering with 100 lines of code
http://www.kevinbeason.com/smallpt/



```
#include <math.h>   // smallpt, a Path Tracer by Kevin Beason, 2008
#include <stdlib.h> // Make : g++ -O3 -fopenmp smallpt.cpp -o smallpt
#include <stdio.h>  //        Remove "-fopenmp" for g++ version < 4.2
struct Vec {        // Usage: time ./smallpt 5000 && xv image.ppm
  double x, y, z;                  // position, also color (r,g,b)
  Vec(double x_=0, double y_=0, double z_=0){ x=x_; y=y_; z=z_; }
  Vec operator+(const Vec &b) const { return Vec(x+b.x,y+b.y,z+b.z); }
  Vec operator-(const Vec &b) const { return Vec(x-b.x,y-b.y,z-b.z); }
  Vec operator*(double b) const { return Vec(x*b,y*b,z*b); }
  Vec mult(const Vec &b) const { return Vec(x*b.x,y*b.y,z*b.z); }
  Vec& norm(){ return *this = *this * (1/sqrt(x*x+y*y+z*z)); }
  double dot(const Vec &b) const { return x*b.x+y*b.y+z*b.z; } // cross:
  Vec operator%(Vec&b){return Vec(y*b.z-z*b.y,z*b.x-x*b.z,x*b.y-y*b.x);}
};
struct Ray { Vec o, d; Ray(Vec o_, Vec d_) : o(o_), d(d_) {} };
enum Refl_t { DIFF, SPEC, REFR };  // material types, used in radiance()
struct Sphere {
  double rad;       // radius
  Vec p, e, c;      // position, emission, color
  Refl_t refl;      // reflection type (DIFFuse, SPECular, REFRactive)
  Sphere(double rad_, Vec p_, Vec e_, Vec c_, Refl_t refl_):
    rad(rad_), p(p_), e(e_), c(c_), refl(refl_) {}
  double intersect(const Ray &r) const { // returns distance, 0 if nohit
    Vec op = p-r.o; // Solve t^2*d.d + 2*t*(o-p).d + (o-p).(o-p)-R^2 = 0
    double t, eps=1e-4, b=op.dot(r.d), det=b*b-op.dot(op)+rad*rad;
    if (det<0) return 0; else det=sqrt(det);
    return (t=b-det)>eps ? t : ((t=b+det)>eps ? t : 0);
  }
};
Sphere spheres[] = {//Scene: radius, position, emission, color, material
  Sphere(1e5, Vec( 1e5+1,40.8,81.6), Vec(),Vec(.75,.25,.25),DIFF),//Left
  Sphere(1e5, Vec(-1e5+99,40.8,81.6),Vec(),Vec(.25,.25,.75),DIFF),//Rght
  Sphere(1e5, Vec(50,40.8, 1e5),     Vec(),Vec(.75,.75,.75),DIFF),//Back
  Sphere(1e5, Vec(50,40.8,-1e5+170), Vec(),Vec(),           DIFF),//Frnt
  Sphere(1e5, Vec(50, 1e5, 81.6),    Vec(),Vec(.75,.75,.75),DIFF),//Botm
  Sphere(1e5, Vec(50,-1e5+81.6,81.6),Vec(),Vec(.75,.75,.75),DIFF),//Top
  Sphere(16.5,Vec(27,16.5,47),       Vec(),Vec(1,1,1)*.999, SPEC),//Mirr
  Sphere(16.5,Vec(73,16.5,78),       Vec(),Vec(1,1,1)*.999, REFR),//Glas
  Sphere(600, Vec(50,681.6-.27,81.6),Vec(12,12,12),  Vec(), DIFF) //Lite
};
inline double clamp(double x){ return x<0 ? 0 : x>1 ? 1 : x; }
inline int toInt(double x){ return int(pow(clamp(x),1/2.2)*255+.5); }
inline bool intersect(const Ray &r, double &t, int &id){
  double n=sizeof(spheres)/sizeof(Sphere), d, inf=t=1e20;
  for(int i=int(n);i--;) if((d=spheres[i].intersect(r))&&d<t){t=d;id=i;}
  return t<inf;
}
Vec radiance(const Ray &r, int depth, unsigned short *Xi){
  double t;                               // distance to intersection
  int id=0;                               // id of intersected object
  if (!intersect(r, t, id)) return Vec(); // if miss, return black
  const Sphere &obj = spheres[id];        // the hit object
  Vec x=r.o+r.d*t, n=(x-obj.p).norm(), nl=n.dot(r.d)<0?n:n*-1, f=obj.c;
  double p = f.x>f.y && f.x>f.z ? f.x : f.y>f.z ? f.y : f.z; // max refl
  if (++depth>5) if (erand48(Xi)<p) f=f*(1/p); else return obj.e; //R.R.
  if (obj.refl == DIFF){                  // Ideal DIFFUSE reflection
    double r1=2*M_PI*erand48(Xi), r2=erand48(Xi), r2s=sqrt(r2);
    Vec w=nl, u=((fabs(w.x)>.1?Vec(0,1):Vec(1))%w).norm(), v=w%u;
    Vec d = (u*cos(r1)*r2s + v*sin(r1)*r2s + w*sqrt(1-r2)).norm();
    return obj.e + f.mult(radiance(Ray(x,d),depth,Xi));
  } else if (obj.refl == SPEC){           // Ideal SPECULAR reflection
    return obj.e + f.mult(radiance(Ray(x,r.d-n*2*n.dot(r.d)),depth,Xi));
  Ray reflRay(x, r.d-n*2*n.dot(r.d));     // Ideal dielectric REFRACTION
  bool into = n.dot(nl)>0;                // Ray from outside going in?
  double nc=1, nt=1.5, nnt=into?nc/nt:nt/nc, ddn=r.d.dot(nl), cos2t;
  if ((cos2t=1-nnt*nnt*(1-ddn*ddn))<0)    // Total internal reflection
    return obj.e + f.mult(radiance(reflRay,depth,Xi));
  Vec tdir = (r.d*nnt - n*((into?1:-1)*(ddn*nnt+sqrt(cos2t))
  double a=nt-nc, b=nt+nc, R0=a*a/(b*b), c = 1-(into?-ddn:tdir.dot(n));
  double Re=R0+(1-R0)*c*c*c*c*c,Tr=1-Re,P=.25+.5*Re,RP=Re/P,TP=Tr/(1-P);
  return obj.e + f.mult(depth>2 ? (erand48(Xi)<P ?   // Russian roulette
    radiance(reflRay,depth,Xi)*RP:radiance(Ray(x,tdir),depth,Xi)*TP) :
    radiance(reflRay,depth,Xi)*Re+radiance(Ray(x,tdir),depth,Xi)*Tr);
}
int main(int argc, char *argv[]){
  int w=1024, h=768, samps = argc==2 ? atoi(argv[1])/4 : 1; // # samples
  Ray cam(Vec(50,52,295.6), Vec(0,-0.042612,-1).norm()); // cam pos, dir
  Vec cx=Vec(w*.5135/h), cy=(cx%cam.d).norm()*.5135, r, *c=new Vec[w*h];
#pragma omp parallel for schedule(dynamic, 1) private(r)       // OpenMP
  for (int y=0; y<h; y++){                        // Loop over image rows
    fprintf(stderr,"\rRendering (%d spp) %5.2f%%",samps*4,100.*y/(h-1));
    for (unsigned short x=0, Xi[3]={0,0,y*y*y}; x<w; x++)   // Loop cols
      for (int sy=0, i=(h-y-1)*w+x; sy<2; sy++)     // 2x2 subpixel rows
        for (int sx=0; sx<2; sx++, r=Vec()){        // 2x2 subpixel cols
          for (int s=0; s<samps; s++){
            double r1=2*erand48(Xi), dx=r1<1 ? sqrt(r1)-1: 1-sqrt(2-r1);
            double r2=2*erand48(Xi), dy=r2<1 ? sqrt(r2)-1: 1-sqrt(2-r2);
            Vec d = cx*( ( (sx+.5 + dx)/2 + x)/w - .5) +
                    cy*( ( (sy+.5 + dy)/2 + y)/h - .5) + cam.d;
            r = r + radiance(Ray(cam.o+d*140,d.norm()),0,Xi)*(1./samps);
          } // Camera rays are pushed ^^^^^ forward to start in interior
          c[i] = c[i] + Vec(clamp(r.x),clamp(r.y),clamp(r.z))*.25;
        }
  }
  FILE *f = fopen("image.ppm", "w");          // Write image to PPM file.
  fprintf(f, "P3\n%d %d %d\n", w, h, 255);
  for (int i=0; i<w*h; i++)
    fprintf(f,"%d %d %d ", toInt(c[i].x), toInt(c[i].y), toInt(c[i].z));
}
```

vector & ray classes

path tracer

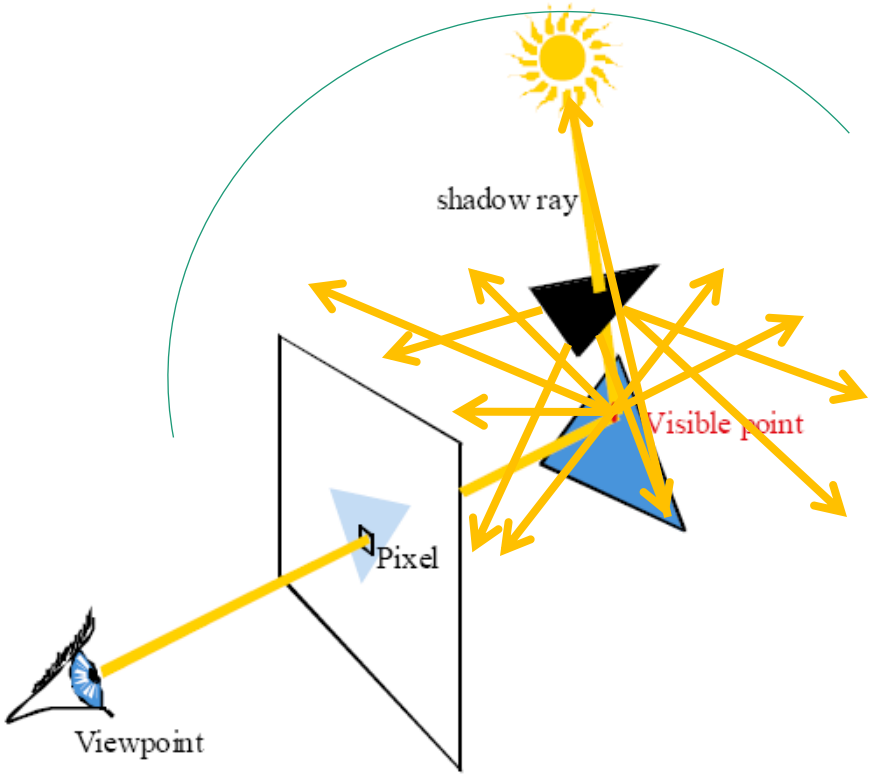
sphere intersection

scene description

eye rays + AA

image output

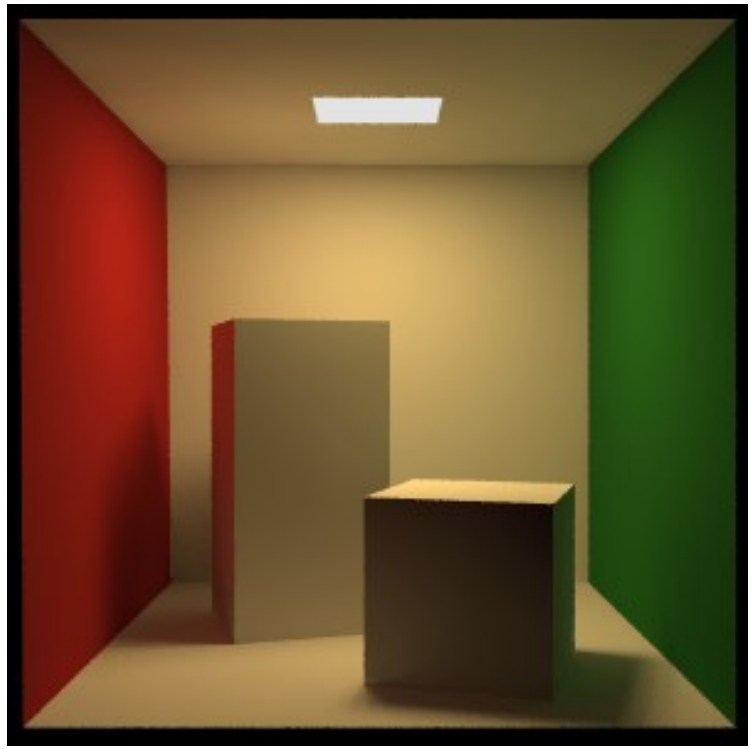
Global Illumination (ASIDE)



- Where does the light come from?
- Can still solve this problem for special cases, or with careful sampling, in both object-order and image-order implementations

Global Illumination (ASIDE)

- Light can bounce off walls to illuminate different surfaces
  - Red reflected light illuminates left side of left object
  - Green reflected light illuminates right side of cube on the right

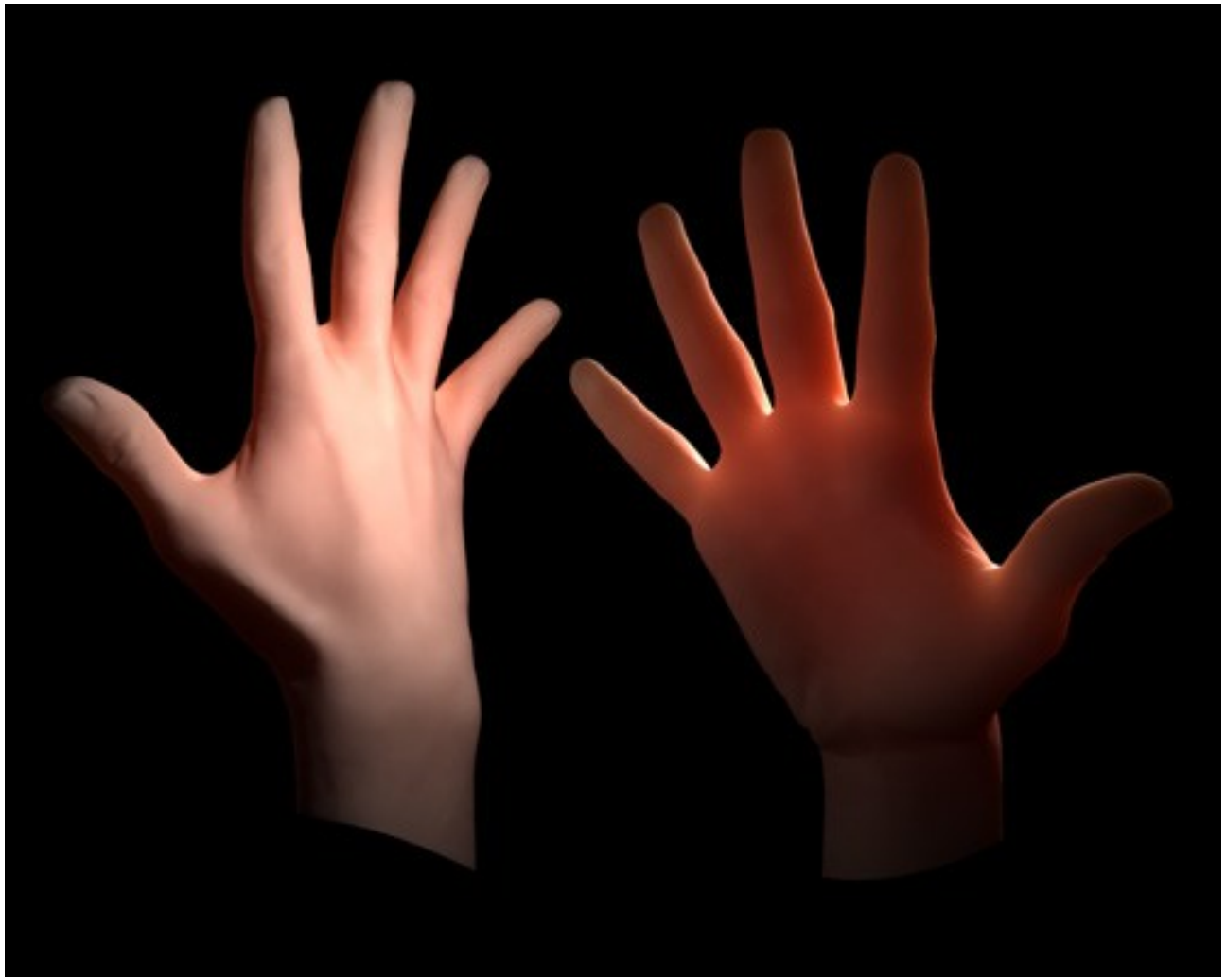


- At right: which is a photo, and which is a rendering?

Cornell PCG

Subsurface Scattering (ASIDE)

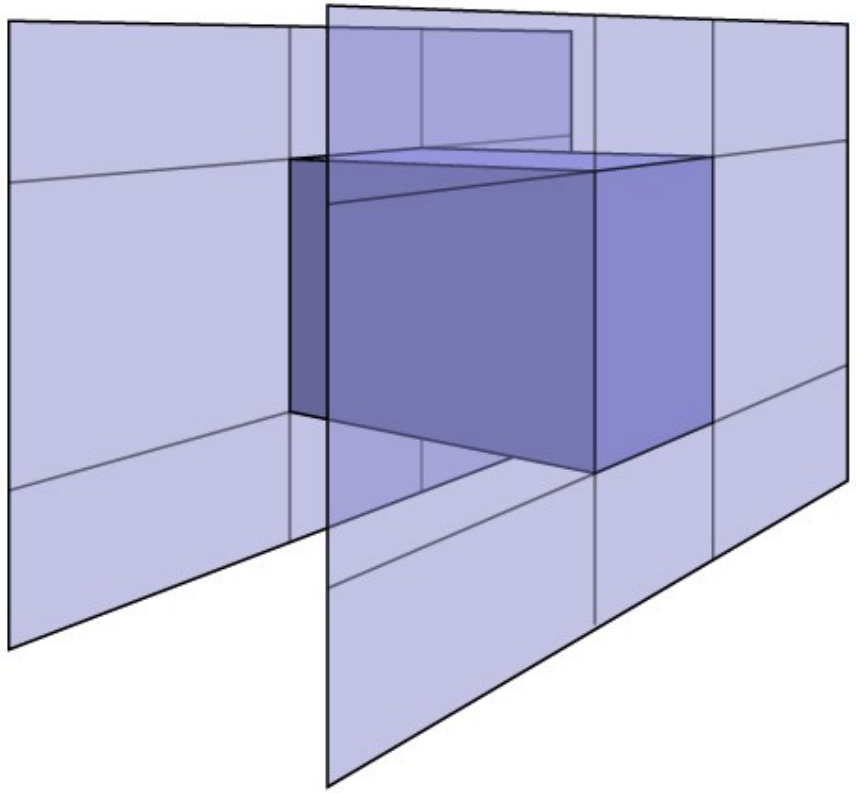
- The full problem is even more complex!!!
  - Light is ***everywhere*** and can even be ***scattered beneath*** the surface of materials
  - Devising a global illumination method that is fast and that correctly models all ***light transport*** is the ultimate problem in rendering.



[spin99, GELATO]

Ray-box intersection

- Could intersect with 6 faces individually
- Better way: box is the intersection of 3 slabs



Ray-slab intersection

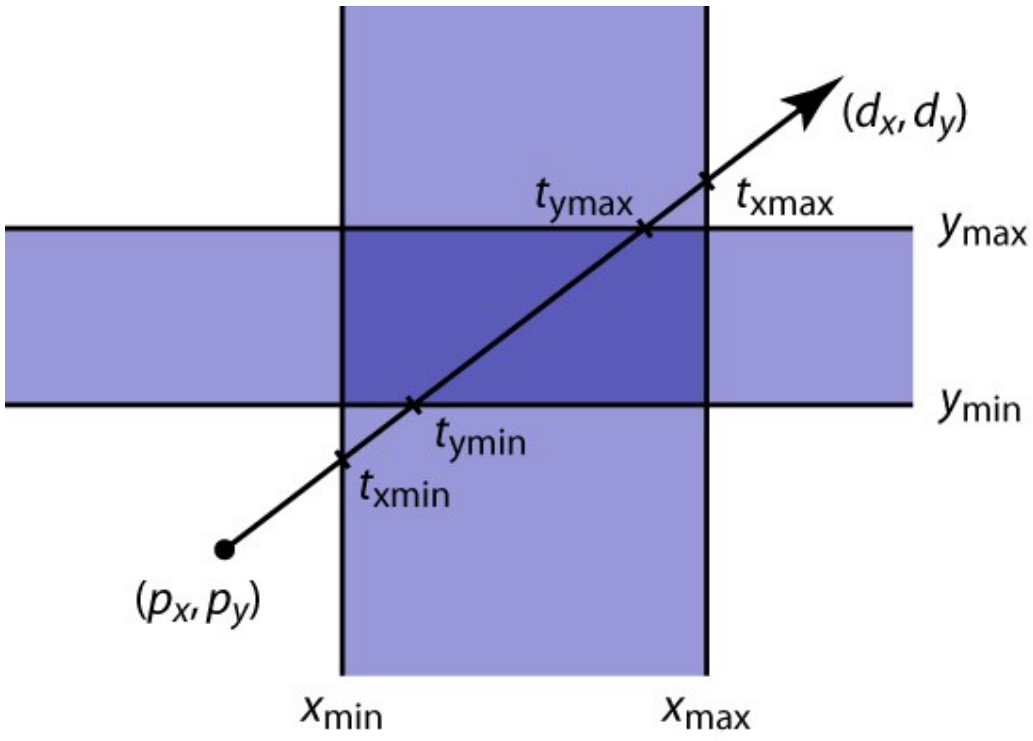
What if $d_x = 0$?
Intersection everywhere if $p_x \in [x_{\min}, x_{\max}]$, otherwise no intersection

- 2D example
- 3D is the same!

$$p_x + t_{x\min}\, d_x = x_{\min}$$
$$t_{x\min} = (x_{\min} - p_x)/d_x$$

$$p_y + t_{y\min}\, d_y = y_{\min}$$
$$t_{y\min} = \left(y_{\min} - p_y\right)/d_y$$

Compute t values where ray crosses each plane



Intersecting intersections

Note that we will later also want the normal associated with the closest side of the slab, e.g., (-1,0,0) for ray 1, and (1,0,0) for ray 2.

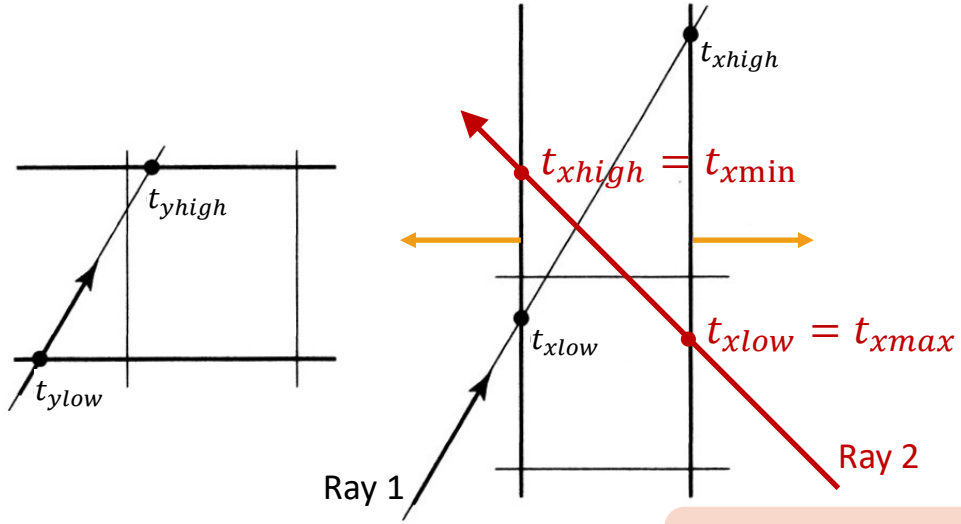
- Each intersection is an interval

  $t_{xlow} = \min(t_{xmin}, t_{xmax})$
  $t_{xhigh} = \max(t_{xmin}, t_{xmax})$

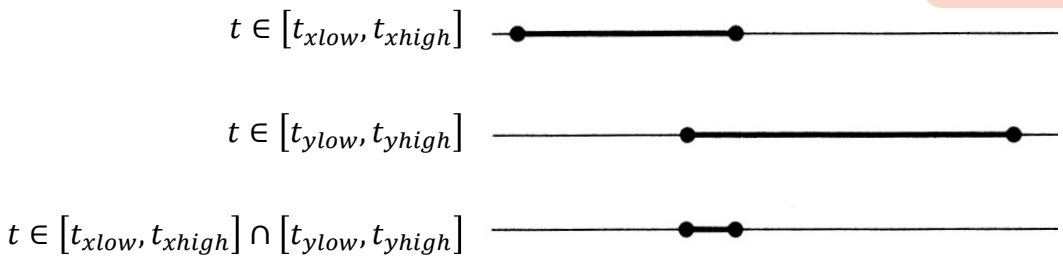
- Want last entry point and first exit point

  $t_{min} = \max(t_{xlow}, t_{ylow})$
  $t_{max} = \min(t_{xhigh}, t_{yhigh})$

- If $t_{max} < t_{min}$ then empty interval, i.e., no intersection



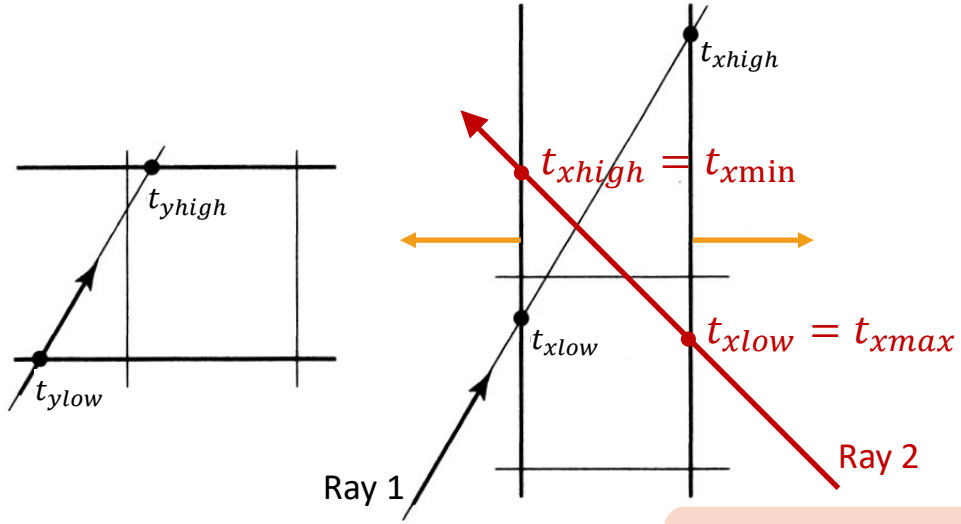
Note example ray 2 coming from opposite direction



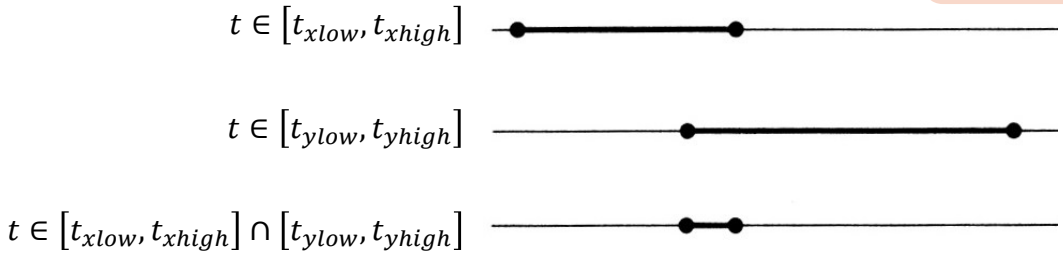
Surface normal needed

Note that we will later also want the normal associated with the closest side of the slab, e.g., (-1,0,0) for ray 1, and (1,0,0) for ray 2.

- Hit result should keep track of surface normal
- If $t = t_{xlow}$ is t_{xmin} then normal is $(-1,0,0)$
- If $t = t_{xlow}$ is t_{xmax} then normal is $(1,0,0)$
- Similarly for other slabs



Note example ray 2 coming from opposite direction



Ray-triangle intersection

- Condition 1: point is on ray

$$\mathbf{r}(t) = \mathbf{p} + t\mathbf{d}$$

- Condition 2: point is on plane

$$(\mathbf{x} - \mathbf{a}) \cdot \mathbf{n} = 0$$

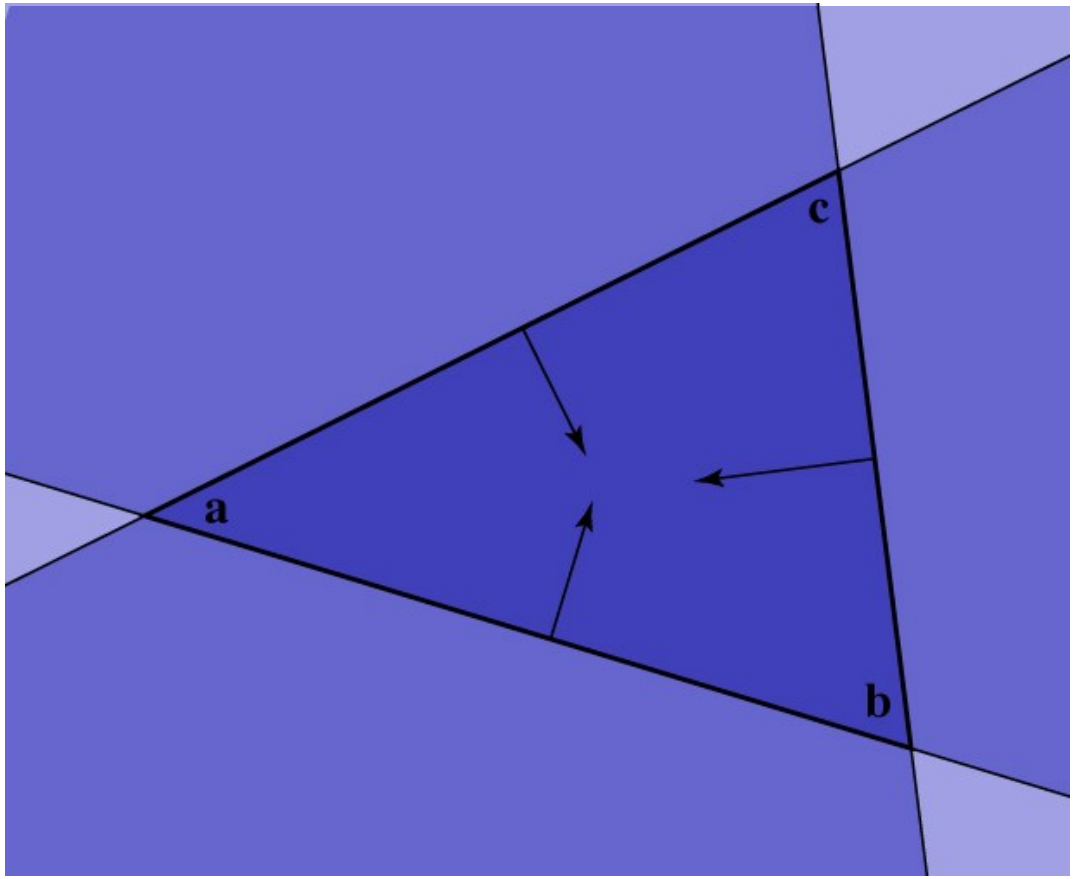
- Condition 3: point is on the inside of all three edges
- First solve 1 and 2 (ray–plane intersection)
  - substitute and solve for t

$$(\mathbf{p} + t\mathbf{d} - \mathbf{a}) \cdot \mathbf{n} = 0 \quad \Leftrightarrow \quad t = \frac{(\mathbf{a} - \mathbf{p}) \cdot \mathbf{n}}{\mathbf{d} \cdot \mathbf{n}}$$

If $\mathbf{d} \cdot \mathbf{n} = 0$ then it is OK to assume no intersection (i.e., ignore this glancing intersection because if the ray were perturbed by epsilon, it would miss the surface entirely)

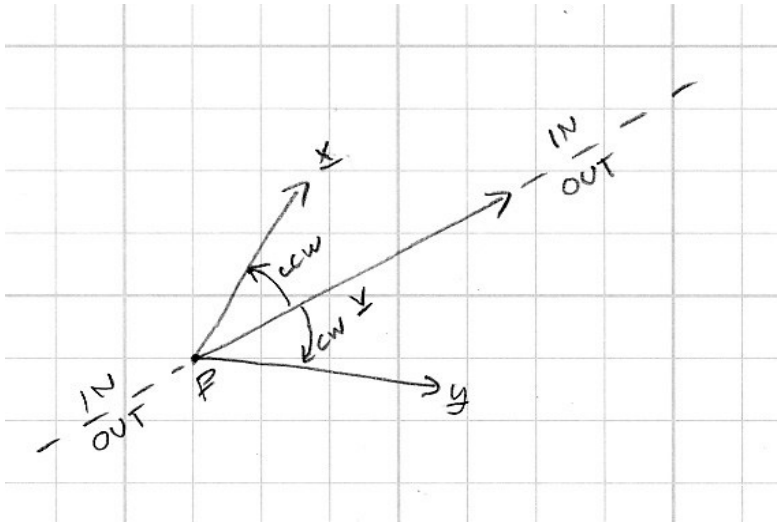
Ray-triangle intersection

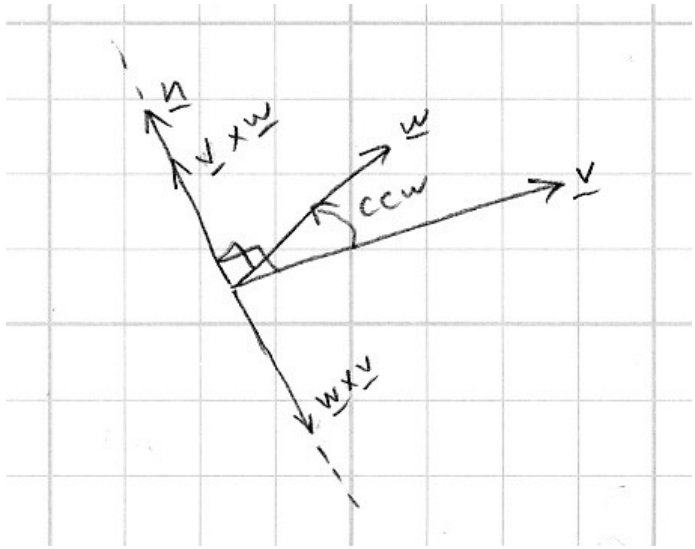
- In plane, the triangle is the intersection of 3 half spaces



Inside-edge test

- Need outside vs. inside
- Reduce to clockwise vs. counterclockwise
  - Vector of edge to vector to **x**
- Use cross product to decide





Ray-triangle intersection

$$(b - a) \times (x - a) \cdot n > 0$$

$$(c - b) \times (x - b) \cdot n > 0$$

$$(a - c) \times (x - c) \cdot n > 0$$

- Can see this as a step toward computing barycentric coordinates (useful as texture coordinates)
- Can also use a similar inside/outside test on non-convex n-gons, by computing the winding number (see CGPP 7.10)

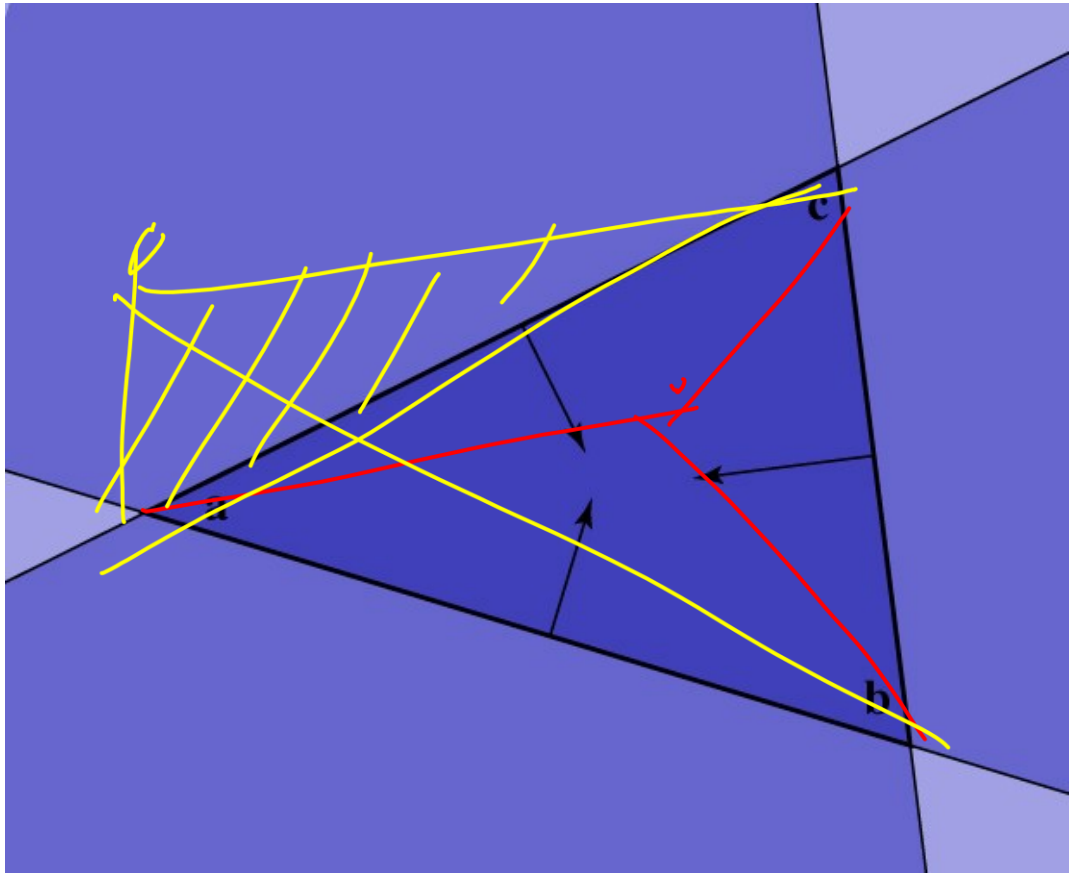
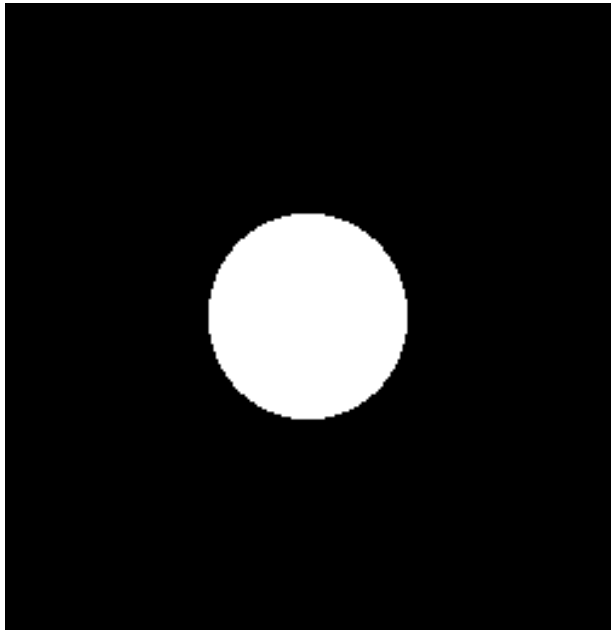


Image so far

- With eye ray generation and sphere intersection

```
Surface s = new Sphere((0.0, 0.0, 0.0), 1.0);
for 0 <= iy < ny {
   for 0 <= ix < nx {
      ray = camera.getRay(ix, iy);
      hitSurface, t = s.intersect(ray, 0, +inf)
      if hitSurface is not null {
           image.set(ix, iy, white);
      }
   }
}
```



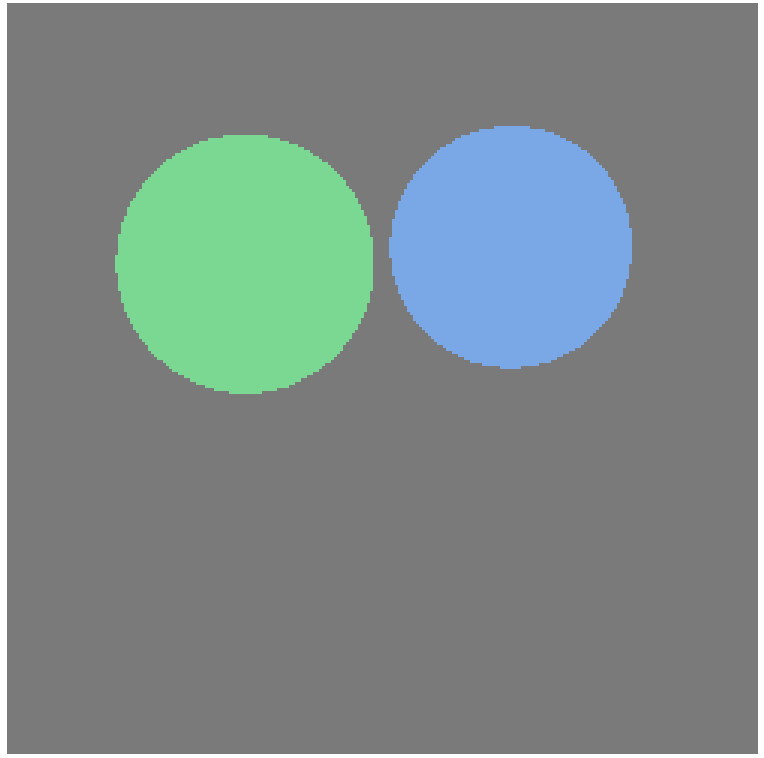
Intersection against many shapes

```
Group.intersect (ray, tMin, tMax) {
    tBest = +inf; firstSurface = null;
    for surface in surfaceList {
        hitSurface, t = surface.intersect(ray, tMin, tBest);
        if hitSurface is not null {
            tBest = t;
            firstSurface = hitSurface;
        }
    }
    return hitSurface, tBest;
}
```

Image so far

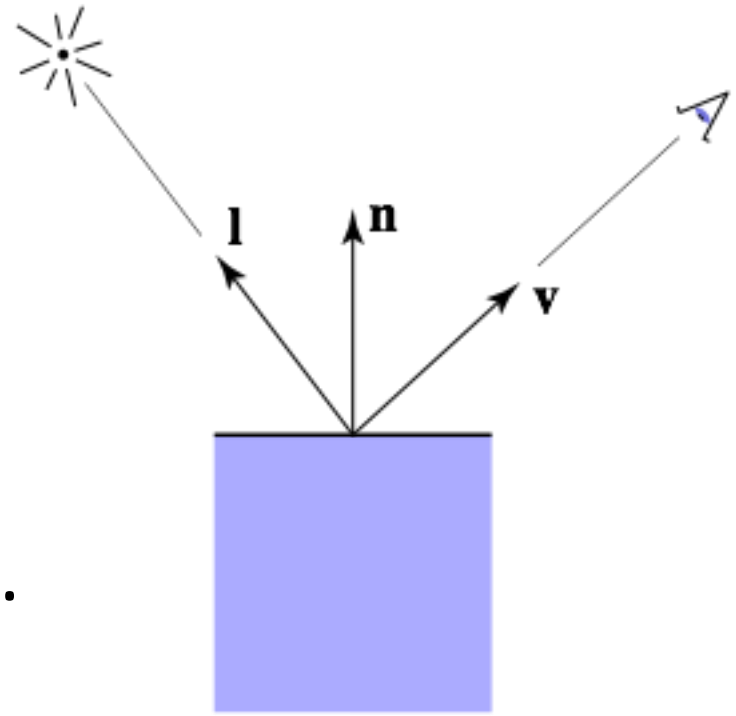
- With eye ray generation and scene intersection

```
for 0 <= iy < ny
    for 0 <= ix < nx {
        ray = camera.getRay(ix, iy);
        c = scene.trace(ray, 0, +inf);
        image.set(ix, iy, c);
    }

…

Scene.trace(ray, tMin, tMax) {
    surface, t = surfs.intersect(ray, tMin, tMax);
    if (surface != null) return surface.color();
    else return black;
}
```



Shading

- Compute light reflected toward camera
- Inputs:
  - eye direction
  - light direction (for each of many lights)
  - surface normal
  - surface parameters (color, shininess, …)
- Exact same equations as seen previously…

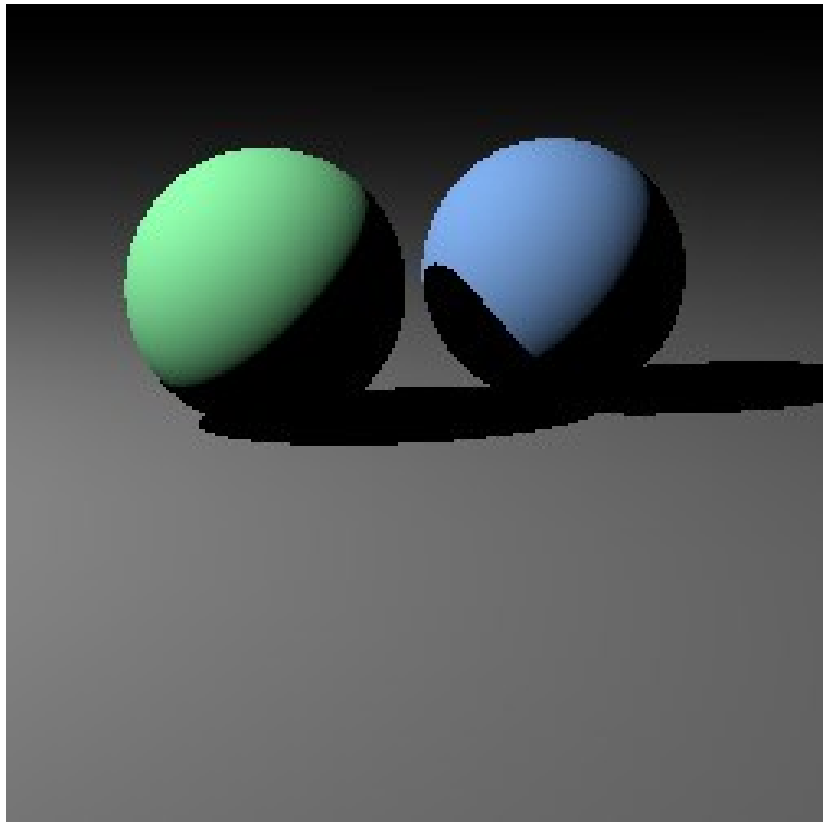


Shadows

- The surface is only illuminated if nothing blocks its view of the light.
- With ray tracing it is easy to check
  - just intersect a ray with the scene!

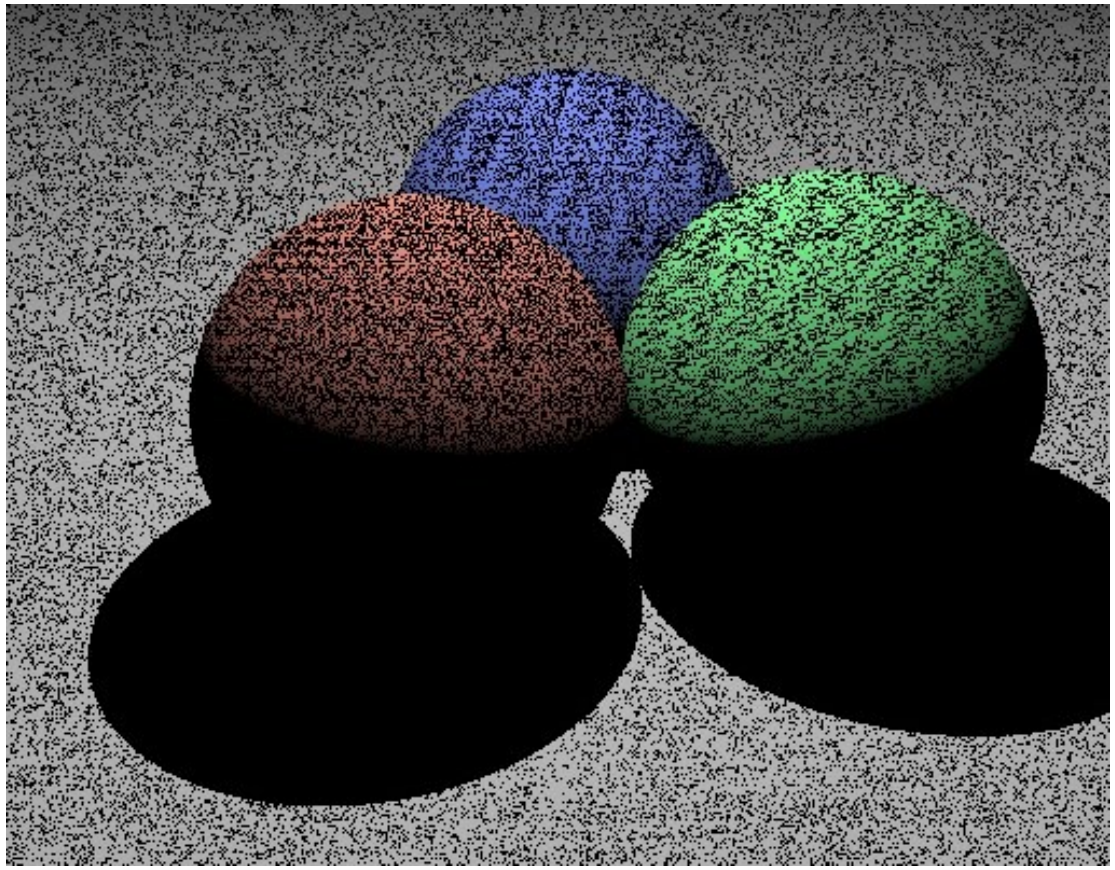
Image so far

```
Surface.shade(ray, point, normal, light) {
    shadRay = (point, light.pos - point);
    if (shadRay not blocked) {
        v = -normalize(ray.direction);
        l = normalize(light.pos - point);
        // compute shading
    }
    return black;
}
```



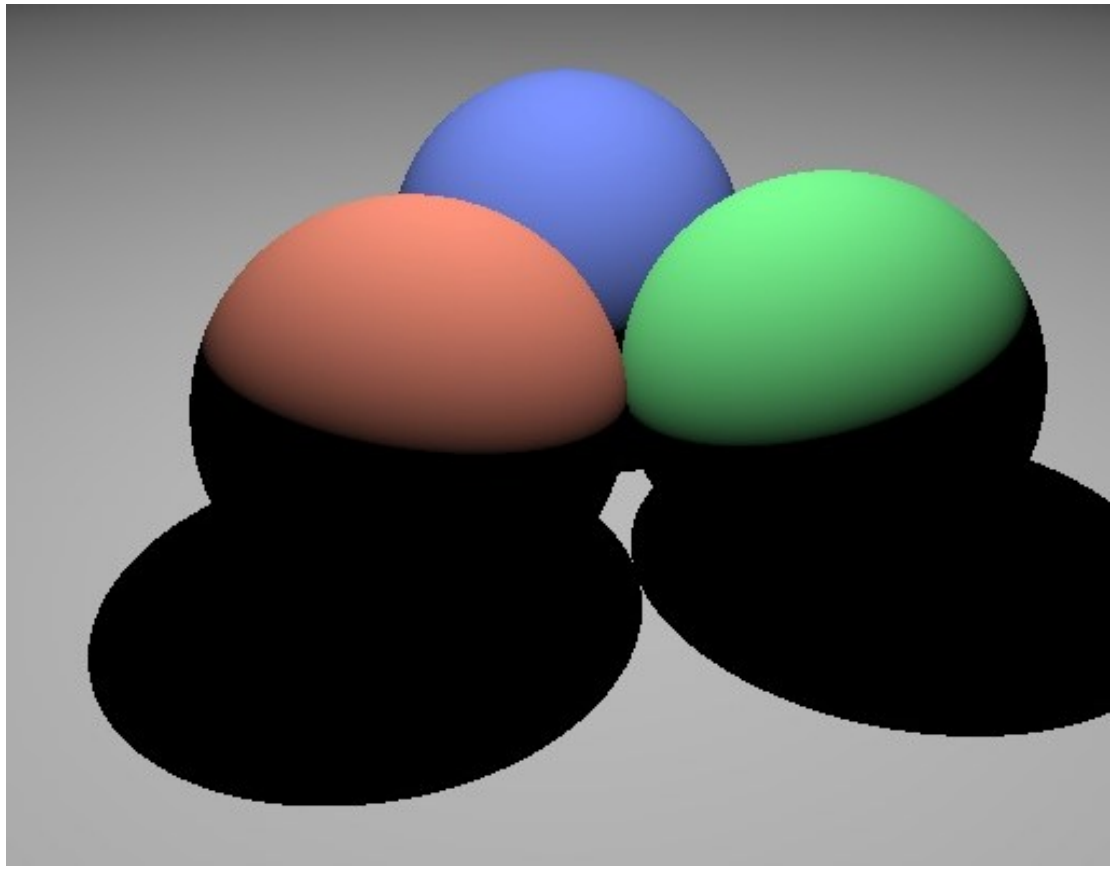
Shadow rounding errors

- Don’t fall victim to one of the classic blunders!
- What’s going on?
  - hint: at what *t* does the shadow ray intersect the surface you’re shading?



Shadow rounding errors

- Solution: shadow rays start a tiny distance from the surface
- Do this by moving the start point, or more easily, by limiting the *t* range



Mirror reflection

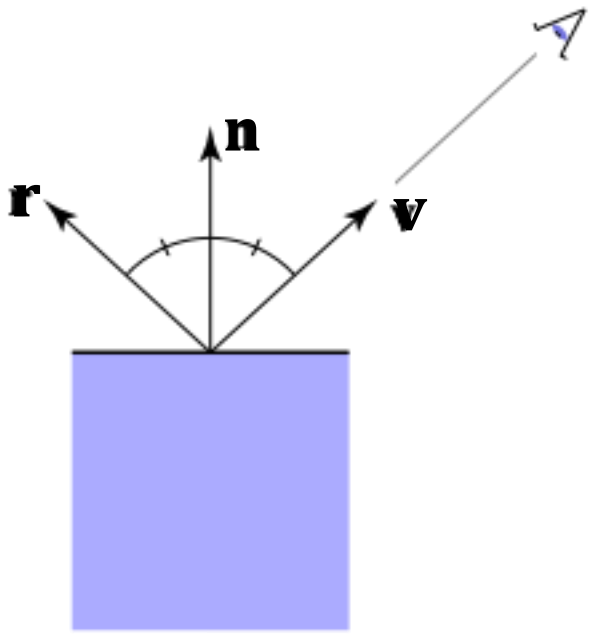
- Consider perfectly shiny surface
  - There isn't a highlight (i.e., no L_s)
  - Instead, there's a reflection of other objects
- Can render this using recursive ray tracing
  - To find out mirror reflection color, ask what color is seen from surface point in reflection direction
  - Already computing reflection direction for Phong?
- "Glazed" material has mirror reflection and diffuse

$$L = L_a + L_d + L_m$$

  - Here L_m is evaluated by tracing a new ray

Mirror reflection

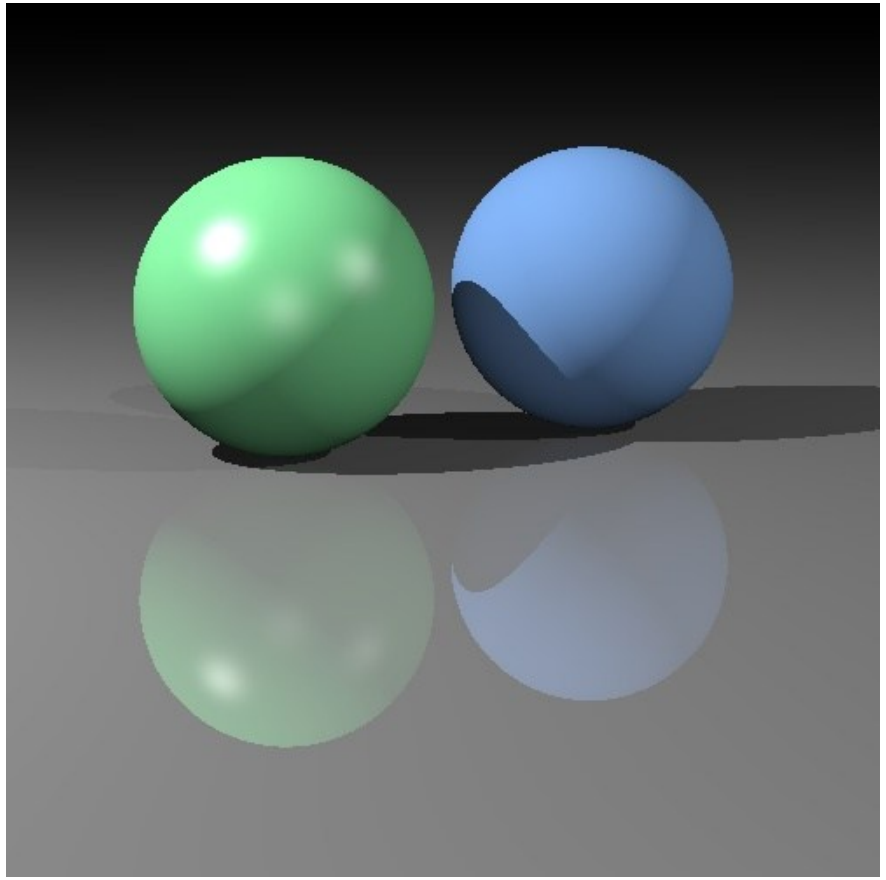
- Intensity depends on view direction
  - Reflects incident light from mirror direction



$$\mathbf{r} = \mathbf{v} + 2\big((\mathbf{n}\cdot\mathbf{v})\mathbf{n} - \mathbf{v}\big)$$
$$= 2(\mathbf{n}\cdot\mathbf{v})\mathbf{n} - \mathbf{v}$$

$$\mathbf{r} = \mathbf{v} + 2(\mathbf{n}(\mathbf{n}^T\mathbf{v}) - \mathbf{v})$$
$$= 2(\mathbf{n}\mathbf{n}^T)\mathbf{v} - \mathbf{v}$$
$$= (2\mathbf{n}\mathbf{n}^T - I)\mathbf{v}$$

Diffuse + mirror reflection (glazed)



(glazed material on floor)

Ray tracer architecture 101

- You want a class called Ray
  - point and direction; evaluate(t)
  - possible: tMin, tMax
- Some things can be intersected with rays
  - individual surfaces
  - groups of surfaces (acceleration goes here)
  - the whole scene
  - make these all subclasses of Surface
  - limit the range of valid *t* values (*e.g.* shadow rays)
- Once you have the visible intersection, compute the color
  - may want to separate shading code from geometry
  - separate class: Material (each Surface holds a reference to one)
  - its job is to compute the color

Architectural practicalities

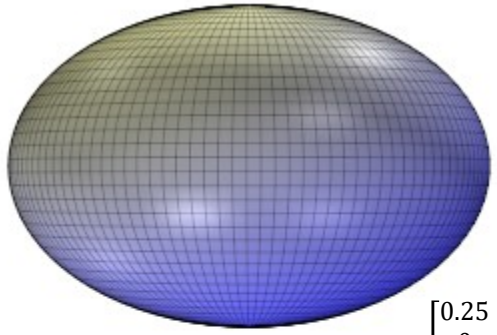
- Return values
  - Surface intersection tends to want to return multiple values
    - Scalar t, surface or shader, normal vector, maybe surface point
  - Typical solution: an *intersection record*
    - A class with fields for all these things
    - Keep track of the intersection record for the closest intersection
- Efficiency
  - In Python, vectorization is a key to being fast, but might not be that easy
    - We will use Taichi for parallelism (Warp-lang also an interesting choice)
  - Shadow rays can be cheaper (any intersection that blocks the light will do, don't need closest)
  - But: "First Get it Right, Then Make it Fast"

Debugging strategies

- Test with small images and simple scenes
- Set breakpoints!!!
  - For example, conditional on a specific pixel
- Make sure your rays are in the correct direction
  - For example, is the ray for the center of the image what you expect it to be?
- Use geometric intuition to reason during debugging!

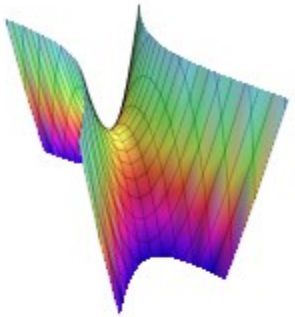
Recall Quadrics

http://en.wikipedia.org/wiki/Quadric

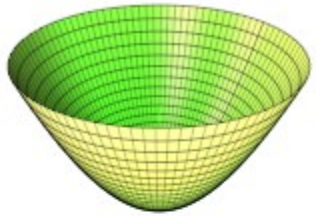


$$\begin{bmatrix} 0.25 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

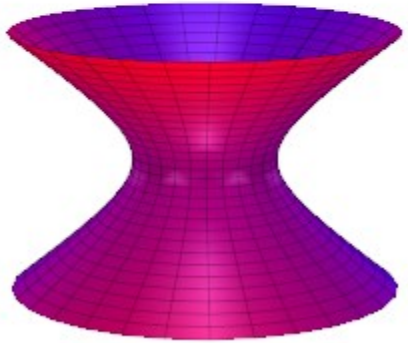
Ellipsoid



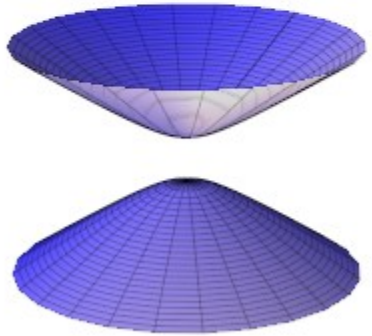
Hyperbolic paraboloid



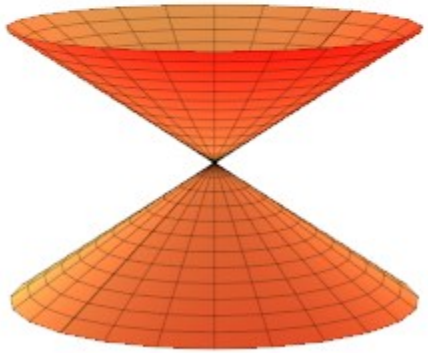
Elliptic paraboloid



Hyperboloid of one sheet



Hyperboloid of two sheets



$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Cone

Quadrics

$$\boldsymbol{x}^T Q \boldsymbol{x} = 0 \qquad Q = \begin{bmatrix} A & -\mathbf{b} \\ -\mathbf{b}^T & c \end{bmatrix} \qquad \boldsymbol{x} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \mathbf{r}(t) = \mathbf{p} + t\mathbf{d}$$ in homogenous coords ($Q \in \mathbb{R}^{4\times 4}$)

- Solution is same as ray sphere intersection.
  - Replace x with ray equation, expand, solve for t
- Given intersection point $\boldsymbol{x}$, what is the normal? Take the gradient!
  But use non-homogeneous coordinates to ignore the 1 in the 4th component of $\boldsymbol{x}$

$$[x\ y\ z]A \begin{bmatrix} x \\ y \\ z \end{bmatrix} - 2\mathbf{b}^T \begin{bmatrix} x \\ y \\ z \end{bmatrix} + c = 0 \qquad A \in \mathbb{R}^{3\times 3} \quad \mathbf{b} \in \mathbb{R}^3 \quad c \in \mathbb{R}$$ in non-homogenous coordinates

$$\nabla F(\boldsymbol{x}) = 2A\boldsymbol{x} - 2\mathbf{b}^T$$

Transforming Quadrics

- What about the implicit surface $\{\, \boldsymbol{v} \mid \boldsymbol{v}^T Q \boldsymbol{v} = 0 \,\}$ when we apply transformation M?
- Can also consider $\{\, \boldsymbol{v} \mid \boldsymbol{v}^T A \boldsymbol{v} + 2\boldsymbol{b}^T \boldsymbol{v} + c = 0 \,\}$ where, A, $\boldsymbol{b}$, and c come from

$$Q = \begin{pmatrix} A_{xx} & A_{xy} & A_{xz} & b_x \\ A_{yx} & A_{yy} & A_{yz} & b_y \\ A_{zx} & A_{zy} & A_{zz} & b_z \\ b_x & b_y & b_z & c \end{pmatrix}$$

  - What happens when we apply an affine transformation $M = \begin{pmatrix} F & t \\ 0 & 1 \end{pmatrix}$?

Transforming Quadrics, answer for $M = \begin{pmatrix} F & t \\ 0 & 1 \end{pmatrix}$

- Note $M^{-1} = \begin{pmatrix} F^{-1} & -F^{-1}t \\ 0 & 1 \end{pmatrix}$
- Want $\{M\,\boldsymbol{v} \mid \boldsymbol{v}^T Q \boldsymbol{v} = 0\}$
- Thus

$$\{\boldsymbol{v} \mid (M^{-1}\boldsymbol{v})^T Q (M^{-1}\boldsymbol{v}) = 0\}$$
$$\{\boldsymbol{v} \mid \boldsymbol{v}^T ({M^{-1}}^T Q M^{-1})\boldsymbol{v} = 0\}$$

- The transformed quadric is described by similarity transform

$$Q_{new} = {M^{-1}}^T Q M^{-1}$$

Transforming Quadrics Example

- Consider a sphere of radius 1, that is $Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
- What happens if you apply a translation by 5 units in y?
- What happens if you scale uniformly by a factor 2?
  - Is this different in comparison to $Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -4 \end{bmatrix}$?
- What happens if you non-uniformly scale x by factor 2?

Review and More Information

- FCG chapter 4
- For Ray-Triangle intersection, note Section 4.4.2 for method based on linear systems and Cramer's rule, but see also Section 2.7 with respect to barycentric coordinates
- For box intersection see FCG 12.3.1